

SELECCION DE IA

Descripción del modelo IA seleccionado y motivo de su elección.

Para hacer un brainstorming inicial, utilicé un agente GPT de OpenAI de React. Le añadí una consulta grande, inicial para el proyecto con la documentación. OpenAI/ChatGPT es muy bueno razonando estructuras y arquitecturas (al igual que Sonnet de Anthropic), y estructurar las herramientas y el planning necesarios.

En cuanto al código día a día, uso Copilot

Prompt 0 - Inicio brainstorming y organización del proyecto:

Yo: Tengo varias peticiones, y te pondré en contexto. Para ello, necesito que adoptes un rol de ingeniero de software y diseñador front-end: Tengo una API creada en PHP con el framework Laravel 12.0, del cual tiene bases de datos. autenticación con Passport v.13.2 y tiene la documentación API creada en Swagger. Te he adjuntado tanto el readme como el archivo swagger para que veas el contexto del proyecto. También te dejo la URL del repositorio: https://github.com/Saikurukaizen/S5_API/tree/dev/S5.01_API_REST/fitbit_api Con toda esta información, falta crear el frontend cliente. Para ello, tendría que ser en base de React. La diferencia es que no tengo que programarla con React en sí, sino mediante IA generativa. Con toda la información proporcionada, realiza, de forma genérica o Brainstorming, los pasos para:

- 1.- Manejar la compatibilidad entre frameworks
- 2.- Valorar el uso de Docker u otra herramienta para evitar la colisión de compatibilidad de versiones, si hiciera falta.
- 3.- De todos los modelos de IA existentes (no sólo de OpenAI), escoger uno o varios modelos que encaje con funciones
vibe coding, para programar en código tal frontend Cliente.
- 4.- Que hayas entendido la documentación proporcionada para ajustarlo a las funciones del frontend y producirla.
En caso contrario, explica qué modelo es más óptimo usar para este caso.
- 5.- Que dicho modelo entienda la contextualización de la compatibilidad backend/frontend.
- 6.- Que habiendo entendido el contexto, el ecosistema y lo que se espera, realices también un to-do list para la planificación.
- 7.- Escoger un system prompt, método ASPECCT y otros usos de prompt engineering para ti u otro modelo y ejecutar tales prompts.

Genérame toda la respuesta, no sólo en modo chat, sino en un pdf descargable.

Su respuesta ha sido un roadmap escrito para MVP descargable en pdf acerca de

las elecciones a tener en cuenta, con recomendaciones.

Después de dockerizar el proyecto y que funcione correctamente, pasa por la elecciones de roles de IA. Le expuse a ChatGPT mis elecciones

Uso de GPT 4o / 4 y Claude Sonnet 3.5 / 4 para la estructura y planificación de validaciones semánticas del frontend, y la coherencia entre componentes. También define el layout visual, wireframes conceptuales y la lógica de interacción. Genera los prompts base de cada componente

Uso de Github Copilot para el código de día a día en el IDE para autocompletado, refactors y tests unitarios.

React Frontend assistant GPT para el asistente contextual de React

Para backend / validaciones API uso Laravel GPT

Agent Claude Sonnet para debugging y errores semánticos

Gracias al roadmap generado, el stack definido y los pasos a seguir, sigue generando especificaciones, punto por punto.

01 – Brainstorming

Frontend React para Fitbit API (Brainstorming, Plan y Prompts)

Autor: React Frontend Assistant (rol: ingeniero de software / diseñador front-end)

Resumen ejecutivo

Este documento contiene un plan detallado y accionable para crear el frontend cliente en React para la API Laravel (Fitbit API) que has provisto. Incluye: compatibilidad entre frameworks, recomendaciones sobre contenedores (Docker), selección de modelos de IA para generar código (vibe-coding), cómo mapear la documentación Swagger a acciones frontend, un **to-do list** de planificación, prompts (system prompt + ASPECTT) para orquestar la generación automatizada del frontend y ejemplos prácticos.

Nota: este plan asume que la API provista expone endpoints con autenticación Bearer (Laravel Passport) y documentación OpenAPI/Swagger funcional (según los archivos que proporcionaste).

0. Fuentes de contexto

README proporcionado (contexto del proyecto, endpoints, setup, testing).

Documentación Swagger completa del API (OpenAPI 3.0, esquemas, ejemplos de request/response).

(Usaste README y Swagger para el análisis del backend y la documentación interactiva.)

1. Manejar la compatibilidad entre frameworks (frontend React ↔ backend Laravel)

Principios

Contrato via OpenAPI: La especificación OpenAPI (Swagger) es la verdad. Generar un cliente TypeScript/JS a partir del `api-docs.json` evita errores manuales y mantiene contrato entre frontend y backend.

Autenticación: Laravel Passport usa tokens Bearer. El frontend debe implementar: login -> recibir token -> guardarlo (preferentemente en memoria + cookie httpOnly o secure storage con atención a XSS), añadir interceptor `Authorization: Bearer {token}` y gestionar expiración/refresh según política (en README indican tokens de 15 días; si no hay refresh token,

renovar con login).

Errores y status codes: Mapear 401/403/422 al UI: redirección a login, mostrar modal de permisos, formularios con validaciones.

Versionado de API: Usa prefijo /api/v1/... desde spec. Mantén una capa de api / services que encapsule llamadas para facilitar cambios futuros.

•

•

1.

2.

3.

4.

1

CORS / CSRF: Configurar CORS en Laravel (ya debería) y en la app React hacer peticiones desde origen correcto. Para producción, configurar APP_URL y APP_ENV en backend y VITE_API_URL en frontend.

Arquitectura recomendada frontend

Lenguaje: TypeScript (mejor tipado con spec OpenAPI)

Bundler: Vite (rápido, moderno)

Routing: React Router v6

Fetching & Cache: TanStack Query (react-query) o RTK Query

State global: Context + hooks para Auth (o Zustand si prefieres) — evitar Redux salvo necesidad compleja

Component library: TailwindCSS (rápido) + librería de componentes (Headless UI / shadcn) si quieres UI consistente

Tests: Vitest + React Testing Library

2. Docker u otra herramienta para evitar colisiones de versiones

¿Por qué usar Docker?

Asegura entorno idéntico entre desarrollador/ci/producción.

Evita conflictos de versiones Node/Python/PHP/DB locales.

Propuesta práctica

Docker para backend: No obligatorio si backend ya corre en XAMPP local, pero recomendable si quieres reproducibilidad y CI.

Docker para frontend: Opcional; usar node:20-alpine para build. Usar docker-compose para orquestar frontend, backend y db local.

Ejemplo de enfoque (docker-compose)

Servicios: frontend (React+Vite), backend (PHP-FPM + Nginx o imagen del proyecto Laravel), db (mysql:8), adminer .

Volúmenes: código fuente montado en frontend/backend para desarrollo.

Beneficios: aislar versiones de Node/NPM, reproducible CI -> puedes usar las mismas imágenes en pipelines.

Alternativas

nvm / volta para manejar versiones Node localmente

devcontainers (VS Code) para reproducir entorno sin Docker en todo el equipo

Recomendación: usar docker-compose para el equipo y devcontainer para onboarding rápido.

5.

•

•

•

•

•

•

•

•

•

•

•

•

•

•

•

•

•

•

•

•

•

•

•

•

•

•

3. Selección de modelos de IA para “vibe coding” (generación de frontend)

Buscamos modelos/servicios que sean buenos generando código, entendiendo especificaciones OpenAPI, y que permitan iteración: debug, tests unitarios, y producción de componentes React+TypeScript.

Candidatos y rol para cada uno

OpenAI (GPT-4o / GPT-4o-mini / GPT-4o-code, si disponible)

Pros: excelente para generación de componentes, tests, y prompts complejos. Buen soporte de few-shot.

Contras: coste, límites de uso; necesitas controlar seguridad y revisar salida.

Anthropic Claude (Claude 3 Haiku / Pro)

Pros: fuerte en instrucciones largas y seguridad; buen razonamiento.

Contras: coste y latencia variable.

Gemini / Google (si disponible públicamente)

Pros: buenas capacidades de generación y contexto largo; integración con infra Google.

Meta / Llama 2 / Llama 3 (via Hugging Face o infraestructura privada)

Pros: opción on-prem / fine-tune; menor coste si self-host.

Contras: menor desempeño out-of-the-box en prompts complejos comparado con top-tier closed models.

Mistral / Mistral-Instruct

Pros: buenos para código, coste competitivo.

Código-specific assistants: GitHub Copilot / Replit AI / Codeium

Pros: están optimizados para asistente de codificación en editores; excelente para generar snippets, completar funciones y tests.

Recomendación práctica (combinada)

Primario (generación de alto nivel, diseño y scaffolding): GPT-4o (o Claude) por su capacidad de entender spec y generar arquitecturas y componentes.

Secundario (iteración rápida en editor): GitHub Copilot / Replit AI para autocompletar y producir componentes de forma interactiva.

On-prem / privacidad: si datos sensibles, usar Llama3/Mistral en infra propia para generación offline.

Motivación: usar un modelo grande para orquestar y diseñar (explicar, mapear endpoints), y copilots para producir código día-a-día.

4. Entendimiento de la documentación provista y ajuste a las funciones del frontend

Lo que entendí (basado en README y Swagger):

API con endpoints versionados `api/v1/*` para Auth, Users, Disciplines, Communities y Stats. filecite turn0file0 filecite turn0file1

Seguridad: autenticación Bearer con Laravel Passport; tokens de 15 días (no se mencionan refresh tokens explícitos). filecite turn0file0

Endpoints públicos: listar disciplinas/communities; endpoints protegidos: crear/editar/eliminar según roles.

OpenAPI disponible: `api-docs.json` y `api-docs.yaml` — ideal para generar cliente TypeScript. filecite turn0file1

Implicaciones para frontend

Generar Cliente API TypeScript: usar `openapi-generator-cli` o `swagger-typescriptapi` para producir modelos y servicios. Esto acelera la integración.

Flujo Auth: implementar hook `useAuth()` y `AuthProvider` que gestione login, logout, me endpoint y almacenamiento del token.

Roles y UI: renderear botones/acciones según rol del me response (User/Admin/Moderator).

Paginación / Filtros: utilizar query params desde spec; un componente `List` reutilizable.

Estadísticas: endpoints de stats se mapearán a dashboards (gráficos, tablas). Usar `Recharts` o `Chart.js` con datos agregados.

Si algo no está cubierto por la spec (por ejemplo, refresh tokens), recomiendo: - Pregunta al backend si existe refresh token. Si no existe, implementar re-login o endpoint custom para refresh.

5. ¿Cómo hacer que el modelo entienda la compatibilidad backend/frontend?

Estrategia de prompt y data inputs

Proveer el OpenAPI JSON (o YAML) como input: el primer paso es pasar la spec al modelo (o una URL) y pedirle que la analice; así el modelo no adivina endpoints.

Proveer el README / notas de implementación (ya hecho) para reglas adicionales (roles, expiración tokens, ejemplos de requests).

Pedir outputs estructurados: generación de `client.ts`, ejemplos de `fetch` / `axios`, y tests unitarios para endpoints claves.

Limitaciones prácticas

Modelos generalmente no ejecutan la spec; conviene usar herramientas automatizadas (OpenAPI Generator) y pedir al modelo que genere wrappers y UI usando esos artefactos.

6. TODO-list (plan de trabajo) — entregable mínimo viable (MVP)

Fase 0 — Preparación

- [] Revisar `api-docs.json` y `api-docs.yaml` (descargar / abrir). (desde `http://localhost:8000/docs/api-docs.json`). filecite turn0file1
- [] Decidir stack: React + Vite + TypeScript + Tailwind + React Router + TanStack Query.
- [] Crear repo `fitbit_frontend` y configurar CI básica (GitHub Actions: `build/test`).

Fase 1 — Scaffolding y Cliente API

- [] Inicializar proyecto Vite + TS
- [] Añadir ESLint, Prettier, Husky, lint-staged
- [] Generar cliente TypeScript con OpenAPI Generator (`typescript-axios` o `typescriptfetch`) apuntando al `api-docs.json` local
- [] Crear carpeta `src/api/` con cliente generado

Fase 2 — Auth y navegación básica

- [] Implementar `AuthProvider` + `useAuth()` (`login`, `logout`, `me`)
- [] Crear `Login` y `Register` pages usando endpoints `/api/v1/login` y `/api/v1/register`
- [] Proteger rutas con `RequireAuth` HOC/component

Fase 3 — CRUD principal

- [] Implementar `Disciplines` list + detail + admin form (`create/update/delete`)
- [] Implementar `Communities` list + detail + `create/join` flows
- [] Implementar `Users` list/profile (`role-based`)

Fase 4 — Dashboard & Stats

- [] Implementar `CommunityStats` dashboard (`ranking`, `percentage`, `summary`)
- [] Visualizaciones con `Recharts`

Fase 5 — Tests y QA

- [] Tests unitarios para components críticos
- [] E2E tests (Playwright) para flows `auth` + `community` CRUD

Fase 6 — Contenedores y despliegue

- [] Añadir `Dockerfile` para frontend y `docker-compose.dev.yaml` para integración con backend
- [] Configurar CI para builds y despliegue estático (Netlify/Vercel/Cloudflare Pages)

7. System prompt, método ASPECCT y prompt engineering (para ejecutar en el modelo)

Propuesta: ASPECCT — acrónimo para diseñar prompts reproducibles

Authentic Context: incluir README + OpenAPI spec + reglas de auth (token duration, roles)

Scope: delimitar exactamente qué generar (ej. `Login` + `AuthProvider` + `useAuth` + tests)

Prompt: el propio texto instructivo con constraints (lenguaje, estilo, estructura)

Examples: incluir 1-2 ejemplos de entrada/salida (few-shot)

Constraints: tamaño del archivo, dependencias permitidas (`axios`, `tanstack-query`), no usar `class` components

Checklist: items que el modelo debe devolver (component TSX, hooks, tests, README de uso)

Test: pedir que genere tests unitarios y casos de uso demostrables

System prompt (para el modelo que generará código)

Eres un asistente especializado en generar frontend React + TypeScript de alta calidad. Tu objetivo es producir código listo para integrar con una API Laravel documentada por OpenAPI 3.0. Lee el OpenAPI JSON proporcionado y genera: 1) cliente TypeScript (si no existe), 2) `AuthProvider` y hook `useAuth`, 3) pantallas `Login` y `Register`, 4) example lists para `Disciplines` y `Communities`, 5) tests unitarios para `Auth` y `Disciplines`. Cumple con las siguientes restricciones: usa Vite, TypeScript, React 18+, React Router v6, TanStack Query para fetching, `axios` (o `fetch`) como cliente HTTP; cadencia de token: Bearer. No uses class components. Devuelve archivos separados (ruta y contenido). Comenta brevemente cada archivo. Si el spec contiene campos extra, mapea model objects con tipos TypeScript.

Ejemplo de prompt de ejecución (few-shot minimal)

Input: api-docs.json (attach or URL), README.md (context),
target=Login+AuthProvider+useAuth

Prompt:

Usa el OpenAPI JSON adjunto. Genera en TypeScript (Vite+React) los siguientes archivos:

- src/api/client.ts (axios instancia con baseUrl = process.env.VITE_API_URL)
- src/context/AuthContext.tsx (AuthProvider que expone user, token, login, logout)
- src/hooks/useAuth.ts (hook)
- src/pages/Login.tsx (formulario con validaciones mínimas)
- tests/Auth.test.tsx (Vitest + React Testing Library)

Requisitos:

- Token guardado en memoria y en cookie secure (ejemplo). Incluye manejo de 401 global (interceptor axios).
- Cada archivo: comentar su propósito y endpoints usados.
- Devuelve el contenido de cada archivo en JSON con campos {path, content}.

Prompt para generar componentes de CRUD a partir de un endpoint

Dada la ruta `GET /api/v1/disciplines` (ver esquema en spec), genera:

- src/pages/Disciplines/List.tsx (listado con paginación básica y link a detail)
- src/pages/Disciplines/Detail.tsx
- src/components/DisciplineForm.tsx (create/update)
- tests/Disciplines.test.tsx

Usa TanStack Query para fetching y query invalidation tras create/update/delete.

Control de calidad automático (post-generation)

Pedir al modelo generar tests unitarios y un script lintfix && vitest.

Pedir un checklist de revisión manual (seguridad, XSS, CSRF, token storage).

8. Pipelines y automatización recomendada

CI: GitHub Actions que ejecute npm ci, npm run lint, npm run test, npm run build.

CD: Deploy a Vercel/Netlify o build estático servido por Nginx dentro de Docker.

IaC: si despliegas backend y frontend juntos, usar infra (Terraform) o AWS S3+CloudFront para frontend.

9. Seguridad y consideraciones finales

Token storage: preferir cookies httpOnly si backend puede crear cookie; si no, guardar token en memoria + refresh strategy para minimizar XSS risk.

HTTPS obligatorio en producción.

Rate limiting / retry policy en cliente.

Validación de inputs en frontend, pero confiar en validaciones del backend.

10. Entregables sugeridos para la primera entrega (MVP)

Repo fitbit_frontend con Vite+TS config

Cliente generado desde OpenAPI (typescript-axios)

Auth flow (Login/Register/ProtectedRoute)

Pages básicas: Disciplines list/detail, Communities list

Dashboard minimal con stats/communities/summary

Dockerfile + docker-compose.dev.yml

Tests unitarios básicos y CI pipeline

Anexos: comandos y snippets útiles

Generar cliente TS (OpenAPI Generator):

```
openapi-generator-cli generate -i http://localhost:8000/docs/api-docs.json -g typescript-axios -o ./src/api/generated --additional-properties=supportsES6=true
```

Vite init:

```
npm create vite@latest fitbit_frontend -- --template react-ts
```

```
cd fitbit_frontend
```

```
npm install
```

Example axios instance (sugerido) - el modelo generador lo producirá en src/api/client.ts.

Observaciones finales

El enfoque híbrido (modelo grande para orquestación + copilots para codificación diaria) ofrece velocidad y control.

El contrato OpenAPI reduce ambigüedad: prioriza generar cliente de API para evitar bugs de integración.

Si quieres, puedo: 1) generar los prompts listos para ejecutar en un modelo (GPT/Claude), 2) crear los archivos scaffold (TSX, hooks, cliente) aquí mismo en el repo, o 3) producir los artefactos Docker y CI.

Fin del documento.

IA WORKFLOW

Planificación (Claude)



Diseño UX/UI (ChatGPT-4o)



Implementación React (React Frontend Assistant + Copilot)



Validación API (Laravel GPT)



Revisión Semántica y Debug (Claude)



Iteración siguiente

Cada ciclo produce artefactos concretos:

/docs/ui-wireframe.md → generado por GPT-4o

/src/routes.tsx → propuesto por React Assistant

/src/components/... → implementado por Copilot

/src/api/client.ts → validado por Laravel GPT

/src/___tests___ → verificado por Claude

Usaré la metodología ASPECCT para coordinar a los modelos:

Prompt 1:

Sigla Función

Ejemplo

A-----Actor / Rol-----"Actúa como diseñador UI experto en dashboards médicos"

S-----Stage / Contexto-----"Estamos en la fase de planificación de la vista principal del panel de actividad del usuario."

P-----Purpose / Objetivo-----"Definir los componentes visuales que mostrarán los datos de Fitbit."

E-----Expectation / Output esperado---"Devuelve un esquema JSON con nombre de componente, tipo y props."

C-----Constraints / Límites-----"Usa TailwindCSS y componentes reutilizables, sin dependencias extra."

C (2)---Checks / Validación-----"Verifica consistencia con los endpoints de la API Swagger."

T-----Tone / Estilo-----"Profesional y académico, claro, con foco en reusabilidad."

Diseño conceptual previo al código

Antes de generar código, se recomienda definir:

□ Estructura de pantallas

Vista Descripción Componentes principales

Login Autenticación OAuth / Password LoginForm, AuthProvider

Dashboard principal Datos Fitbit (ritmo, sueño, pasos, calorías)

DashboardLayout, ActivityChart, SummaryCard

Historial / Reports Visualización por fechas HistoryTable, DatePicker, ExportButton

Perfil de usuario Datos personales y configuración UserProfileForm,

AvatarUpload, SettingsPanel

□ Integración con API Laravel

Cada componente deberá mapearse con un endpoint del Swagger:

/api/login → AuthService.login()

/api/fitbit/data → DataService.getAll()

/api/user/profile → UserService.getProfile()

🕒 5 Próximos pasos concretos

(Diseño visual) → ChatGPT-4o: genera wireframes conceptuales (layout, componentes).

(Estructura de rutas y componentes) → Claude: define árbol de carpetas y arquitectura.

(Implementación) → React Frontend Assistant + Copilot: generan los archivos React + TS.

(Validación API) → Laravel GPT: verifica compatibilidad Swagger ↔ servicios.

(Depuración semántica) → Claude: optimiza y refactoriza.

🌀 6 Qué deberíamos hacer ahora (siguiente paso inmediato)

Tú tienes razón: antes de generar código, toca diseñar la interfaz base (UI/UX).

Entonces, el siguiente paso es:

□ Definir la estructura visual y funcional del frontend (pantallas + navegación + estilo general).

Podemos hacerlo con ChatGPT-4o (modo diseño) o directamente aquí — yo puedo producir el wireframe textual (o imagen si quieres).

Ya que ChatGPT no es gratuito, y los tokens son limitados durante el día, le he pedido a Sonnet 4 que haga tal un mock-up, con la documentación y el contexto del chat anterior. El resultado ha sido increíble.

Prompt 2 - Aquí tienes toda la información sobre un proyecto de API REST ya definido al que hacer la parte cliente Frontend (tienes toda la especificación y el roadmap en el pdf). Te enviaré además los últimos updates de prompts de consulta. A partir de aquí, y con todo el contexto, siendo tú un diseñador de software, escoge un wireframe textual de diseño UX/UI del proyecto Fitbit, con una imagen de muestra. Coge el esqueleto que la documentación Swagger y Readme te ha proporcionado, y généralo. Que sea visualmente agradable y un diseño adaptado a una aplicación de fitness deportivo. Esto será la base para la posterior implementación de React + TS

02 – Fitbit UX/UI wireframe – Documentación Completa

? Wireframe y Diseño UX/UI - Fitbit Fitness Tracker

****Autor:**** React Frontend Assistant

****Proyecto:**** Fitbit API - Frontend Cliente React + TypeScript

****Fecha:**** Enero 2025

****Versión:**** 1.0

? Tabla de Contenidos

1. [Resumen Ejecutivo](#resumen-ejecutivo)
2. [Arquitectura de Navegación](#arquitectura-de-navegación)
3. [Wireframes por Pantalla](#wireframes-por-pantalla)
4. [Especificaciones Técnicas](#especificaciones-técnicas)
5. [Mapeo de Componentes React](#mapeo-de-componentes-react)
6. [Integración con API](#integración-con-api)
7. [Plan de Implementación](#plan-de-implementación)

? Resumen Ejecutivo

Este documento contiene el diseño completo de la interfaz de usuario (UX/UI) para el frontend del proyecto ****Fitbit Fitness Tracker****, una aplicación web desarrollada en React + TypeScript + Vite que consume la API REST Laravel documentada.

Objetivos del Diseño

- ✓ ****Moderno y deportivo****: Paleta energética con gradientes vibrantes
- ✓ ****Funcional y claro****: Navegación intuitiva según roles de usuario
- ✓ ****Modular****: Componentes React reutilizables y escalables
- ✓ ****Responsive****: Diseño adaptable a mobile, tablet y desktop

- ✓ ****Accesible****: Cumpliendo estándares WCAG 2.1

Stack Tecnológico Frontend

```
```\n- React 18+\n- TypeScript 5+\n- Vite (bundler)\n- TailwindCSS (styling)\n- React Router v6 (navegación)\n- TanStack Query (data fetching)\n- Recharts (visualizaciones)\n- Axios (HTTP client)\n```
```

---

## ## 🏗️ Arquitectura de Navegación

### ### Mapa de Flujo de Usuario



\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

### ### Rutas de la Aplicación

Ruta	Acceso	Componente	Descripción
-----	-----	-----	-----
<code>/login</code>	Público	<code>LoginPage</code>	Autenticación
<code>/register</code>	Público	<code>RegisterPage</code>	Registro nuevo usuario
<code>/</code>	Auth	<code>DashboardPage</code>	Dashboard principal
<code>/profile</code>	Auth	<code>ProfilePage</code>	Perfil del usuario
<code>/communities</code>	Auth	<code>CommunitiesPage</code>	Listado comunidades
<code>/communities/:id</code>	Auth	<code>CommunityDetailPage</code>	Detalle comunidad
<code>/disciplines</code>	Auth	<code>DisciplinesPage</code>	Listado disciplinas
<code>/users</code>	Admin/Mod	<code>UsersPage</code>	Gestión usuarios
<code>/stats</code>	Admin/Mod	<code>StatsPage</code>	Dashboard estadísticas

111

## ## ? Wireframes por Pantalla

### 1 LOGIN & REGISTER (Público)

```

? FITBIT FITNESS TRACKER
Tu comunidad deportiva digital

[Email]
[Password]

[? INICIAR SESIÓN]

¿No tienes cuenta?
[Regístrate aquí]

? Credenciales demo:
User: doom@user.com / 666
Admin: lux@admin.com / 8000

```

## **\*\*Componentes React:\*\***

- `LoginForm.tsx`
- `RegisterForm.tsx`
- `AuthLayout.tsx`

## **\*\*Endpoints API:\*\***

```
- `POST /api/v1/login`
- `POST /api/v1/register`
```

```

Validaciones:

```

- Email válido (formato)
- Password mínimo 6 caracteres
- Manejo de errores 401/422

— — —

### 2 DASHBOARD PRINCIPAL (Autenticado)



? FITBIT | Dashboard | Mi Perfil | [Logout] ? Marc

Hola, Marc Sanchez ? [Role: User]

**? MIS ESTADÍSTICAS**

?	COMUNID		?	DISCIPL		?	ACTIVOS
---	---------	--	---	---------	--	---	---------

3

5

14:

Unidas

## Favoritas

## Miembros

**? MIS COMUNIDADES**

? Running Club Barcelona [156 miembros] [Ver]

? Cycling Masters [89 miembros] [Ver]

? Swimming Team Pro [67 miembros] [Ver]

## ? DISCIPLINAS DISPONIBLES

?

?

?



## Running

## Cycling

Swimming

## Football

[+Más]

A dark, abstract graphic with horizontal lines and a small text element. The text "[+ Unirse a nueva comunidad]" is visible on the left side, followed by a long horizontal bar.

```

Componentes React:
- `DashboardLayout.tsx`
- `NavBar.tsx`
- `StatsCards.tsx`
- `CommunityList.tsx`
- `DisciplineGrid.tsx`
- `WelcomeHeader.tsx`

```

- `DashboardLayout.tsx`
- `NavBar.tsx`
- `StatsCards.tsx`
- `CommunityList.tsx`
- `DisciplineGrid.tsx`
- `WelcomeHeader.tsx`

```
Endpoints API:
- `GET /api/v1/me`
- `GET /api/v1/communities`
- `GET /api/v1/disciplines`
- `GET /api/v1/stats/communities/summary`
```

```
- `GET /api/v1/me`
- `GET /api/v1/communities`
- `GET /api/v1/disciplines`
- `GET /api/v1/stats/communities/summary`
```

```
Features:
- Visualización de estadísticas personales
- Acceso rápido a comunidades del usuario
- Exploración de disciplinas disponibles
- Acciones contextuales según rol
```

- Visualización de estadísticas personales
- Acceso rápido a comunidades del usuario
- Exploración de disciplinas disponibles
- Acciones contextuales según rol

11

### ### 3. GESTIÓN DE COMUNIDADES (Admin/Moderator)

[illegible]

Running BCN	? Running	156	[ ] [? ???]
Cycling Masters	? Cycling	89	[ ] [? ???]
Swim Team Pro	? Swimming	67	[ ] [? ???]
Football League	⚽ Football	45	[ ] [? ???]
...	...	...	...

## ? ESTADÍSTICAS RÁPIDAS

- Total comunidades: 24
- Total miembros: 1,240
- Disciplina más popular: Running (35%)

## \*\*Componentes React:\*\*

- `CommunityTable.tsx`
- `CommunityForm.tsx` (modal)
- `SearchBar.tsx`
- `FilterDropdown.tsx`
- `QuickStats.tsx`
- `ActionButtons.tsx`

## \*\*Endpoints API:\*\*

- `GET /api/v1/communities`
- `POST /api/v1/communities`
- `PUT /api/v1/communities/{id}`
- `DELETE /api/v1/communities/{id}`
- `GET /api/v1/stats/communities/ranking`

## \*\*Funcionalidades:\*\*

- CRUD completo de comunidades
- Búsqueda en tiempo real
- Filtrado por disciplina
- Confirmación de eliminación
- Validación de permisos según rol

## ### 4. PERFIL DE USUARIO

? FITBIT | ? MI PERFIL

MARC SANCHEZ

doom@user.com

[AVATAR]

? Role: User

? Miembro desde: Enero 2025

?

INFORMACIÓN PERSONAL

Nombre:

Marc Sanchez

[Editar]

Email:

doom@user.com

[Editar]

Password:

.....

[Cambiar]

?

MIS COMUNIDADES (3)

• Running Club Barcelona

[Salir]

• Cycling Masters

[Salir]

• Swimming Team Pro

[Salir]

[? Guardar Cambios]

[? Eliminar Cuenta]

## **\*\*Componentes React:\*\***

- `UserProfile.tsx`
- `AvatarUpload.tsx`
- `ProfileForm.tsx`
- `UserCommunitiesList.tsx`
- `PasswordChangeModal.tsx`

## **\*\*Endpoints API:\*\***

- `GET /api/v1/users/{id}`
- `PUT /api/v1/users/{id}`
- `DELETE /api/v1/users/{id}`

## **\*\*Validaciones:\*\***

- Edición solo del propio perfil (usuarios normales)
- Admin/Mod pueden editar cualquier perfil
- Confirmación para eliminar cuenta
- Validación de contraseña actual antes de cambiar

---

## **### 5▯ ESTADÍSTICAS (Admin/Moderator)**

---

? FITBIT		? ESTADÍSTICAS	
? RANKING DE COMUNIDADES (Top 5)			
1.	? Running BCN	[████████████████████]	156
2.	? Cycling Masters	[██████████████████]	89
3.	? Swimming Team	[██████████████████]	67
4.	⚽ Football League	[██████████████████]	45
5.	?🏋️🏊🚴 Gym Warriors	[██████████████████]	34
? DISTRIBUCIÓN POR DISCIPLINA			
[Gráfico de Pastel]			
? Running: 35%			
? Cycling: 28%			
? Swimming: 20%			
⚽ Football: 12%			
?🏋️🏊🚴 Otros: 5%			
? RESUMEN GENERAL			
• Total usuarios: 1,240			
• Total comunidades: 24			
• Total disciplinas: 12			
• Promedio miembros/comunidad: 51.6			



## **\*\*Componentes React:\*\***

- `StatsLayout.tsx`
- `RankingChart.tsx` (Recharts)
- `DisciplineDistributionChart.tsx` (Recharts)
- `SummaryMetrics.tsx`
- `ExportDataButton.tsx`

## **\*\*Endpoints API:\*\***

- `GET /api/v1/stats/communities`
- `GET /api/v1/stats/communities/ranking`
- `GET /api/v1/stats/communities/percentage`
- `GET /api/v1/stats/communities/summary`
- `GET /api/v1/stats/communities/by-discipline`

## **\*\*Features:\*\***

- Gráficos interactivos con Recharts
- Actualización en tiempo real
- Exportación de datos (CSV/PDF)
- Filtros por período de tiempo

---

## **## ? Especificaciones Técnicas**

### **### ? Paleta de Colores**

```
``typescript
// theme/colors.ts
export const colors = {
 primary: {
 gradient: 'linear-gradient(135deg, #667eea 0%, #764ba2 100%)',
 solid: '#667eea',
 light: '#a5b4fc',
 dark: '#5568d3'
 },
 secondary: {
 gradient: 'linear-gradient(135deg, #f093fb 0%, #f5576c 100%)',
 solid: '#f5576c',
 light: '#fbb6ce'
 },
 success: '#48bb78',
 warning: '#f6ad55',
 error: '#fc8181',
 neutral: {
 bg: '#f7fafc',
 card: 'ffffff',
 text: '#2d3748',
```

```
 textLight: '#718096',
 border: '#e2e8f0'
 }
}
```
```

? Tipografía

```
```typescript
// theme/typography.ts
export const typography = {
 fontFamily: "'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif",
 sizes: {
 h1: '32px', // Dashboard titles
 h2: '24px', // Section titles
 h3: '18px', // Card titles
 body: '16px', // Regular text
 small: '14px', // Secondary text
 xs: '12px' // Labels, badges
 },
 weights: {
 regular: 400,
 medium: 500,
 semibold: 600,
 bold: 700
 }
}
```
```

? Breakpoints (Responsive)

```
```typescript
// theme/breakpoints.ts
export const breakpoints = {
 mobile: '320px',
 tablet: '768px',
 desktop: '1024px',
 wide: '1440px'
}
```

```
// Tailwind config
module.exports = {
 theme: {
 screens: {
 'sm': '640px',
 'md': '768px',
 'lg': '1024px',
 'xl': '1280px',
 '2xl': '1536px',
 }
 }
}
```

```
}
``
```

### ### ? Animaciones y Transiciones

```
```css  
/* Transiciones suaves */  
.smooth-transition {  
  transition: all 0.3s ease-in-out;  
}  
  
/* Hover effects */  
.card-hover:hover {  
  transform: translateY(-5px);  
  box-shadow: 0 12px 32px rgba(102, 126, 234, 0.3);  
}  
  
/* Loading states */  
@keyframes pulse {  
  0%, 100% { opacity: 1; }  
  50% { opacity: 0.5; }  
}  
``
```

```
---
```

? Mapeo de Componentes React

Estructura de Carpetas

```
``  
src/  
├── api/  
│   ├── client.ts           # Axios instance  
│   ├── generated/         # OpenAPI generated  
│   └── services/  
│       ├── auth.service.ts  
│       ├── user.service.ts  
│       ├── community.service.ts  
│       ├── discipline.service.ts  
│       └── stats.service.ts  
├── components/  
│   ├── common/  
│   │   ├── Button.tsx  
│   │   ├── Input.tsx  
│   │   ├── Card.tsx  
│   │   ├── Modal.tsx  
│   │   ├── LoadingSpinner.tsx  
│   │   └── ErrorBoundary.tsx  
└──
```

```
| | — layout/
| |   | — NavBar.tsx
| |   | — Sidebar.tsx
| |   | — Footer.tsx
| |   | — DashboardLayout.tsx
| |
| | — auth/
| |   | — LoginForm.tsx
| |   | — RegisterForm.tsx
| |   | — AuthProvider.tsx
| |
| | — community/
| |   | — CommunityCard.tsx
| |   | — CommunityList.tsx
| |   | — CommunityForm.tsx
| |   | — CommunityTable.tsx
| |   | — MembersList.tsx
| |
| | — discipline/
| |   | — DisciplineCard.tsx
| |   | — DisciplineGrid.tsx
| |   | — DisciplineForm.tsx
| |
| | — user/
| |   | — UserProfile.tsx
| |   | — UserTable.tsx
| |   | — AvatarUpload.tsx
| |   | — ProfileForm.tsx
| |
| | — stats/
| |   | — StatsCards.tsx
| |   | — RankingChart.tsx
| |   | — DistributionChart.tsx
| |   | — SummaryMetrics.tsx
| |
| | — pages/
| |   | — LoginPage.tsx
| |   | — RegisterPage.tsx
| |   | — DashboardPage.tsx
| |   | — ProfilePage.tsx
| |   | — CommunitiesPage.tsx
| |   | — CommunityDetailPage.tsx
| |   | — DisciplinesPage.tsx
| |   | — UsersPage.tsx
| |   | — StatsPage.tsx
| |
| | — hooks/
| |   | — useAuth.ts
| |   | — useCommunities.ts
| |   | — useDisciplines.ts
| |   | — useUsers.ts
```

```

├── useStats.ts
├──
├── contexts/
│   ├── AuthContext.tsx
│   └──
├── utils/
│   ├── validators.ts
│   ├── formatters.ts
│   └── constants.ts
├──
├── types/
│   ├── auth.types.ts
│   ├── user.types.ts
│   ├── community.types.ts
│   └── api.types.ts
├──
├── theme/
│   ├── colors.ts
│   ├── typography.ts
│   └── breakpoints.ts
└──
```

```

### ### Componentes Clave y Responsabilidades

| Componente        | Responsabilidad                    | Props Principales        |
|-------------------|------------------------------------|--------------------------|
| `AuthProvider`    | Manejo de autenticación global     | `children`               |
| `useAuth`         | Hook para acceder al contexto auth | -                        |
| `NavBar`          | Navegación principal               | `user`, `onLogout`       |
| `DashboardLayout` | Layout principal con sidebar       | `children`               |
| `CommunityCard`   | Tarjeta de comunidad               | `community`, `onJoin`    |
| `CommunityForm`   | Formulario CRUD comunidades        | `community?`, `onSubmit` |
| `StatsCards`      | Tarjetas de estadísticas           | `stats`                  |
| `RankingChart`    | Gráfico de ranking (Recharts)      | `data`                   |

---

### ## ? Integración con API

#### ### Configuración del Cliente HTTP

```

```typescript
// src/api/client.ts
import axios from 'axios';

const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:8000/api/v1',
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
  },
},

```

```
});
```

```
// Interceptor para añadir token
```

```
apiClient.interceptors.request.use((config) => {  
  const token = localStorage.getItem('token'); // 0 desde contexto  
  if (token) {  
    config.headers.Authorization = `Bearer ${token}`;  
  }  
  return config;  
});
```

```
// Interceptor para manejar errores
```

```
apiClient.interceptors.response.use(  
  (response) => response,  
  (error) => {  
    if (error.response?.status === 401) {  
      // Redirigir a login  
      window.location.href = '/login';  
    }  
    return Promise.reject(error);  
  }  
);
```

```
export default apiClient;
```

```
``
```

```
### Servicios por Módulo
```

```
``typescript
```

```
// src/api/services/auth.service.ts
```

```
import apiClient from '../client';
```

```
export const authService = {  
  login: async (email: string, password: string) => {  
    const { data } = await apiClient.post('/login', { email, password });  
    return data;  
  },  
  register: async (userData: RegisterData) => {  
    const { data } = await apiClient.post('/register', userData);  
    return data;  
  },  
  logout: async () => {  
    const { data } = await apiClient.post('/logout');  
    return data;  
  },  
  me: async () => {  
    const { data } = await apiClient.get('/me');  
    return data;  
  }  
};
```

```
}  
};  
``
```

Hooks con TanStack Query

```
``typescript  
// src/hooks/useCommunities.ts  
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';  
import { communityService } from '@api/services/community.service';  
  
export const useCommunities = () => {  
  return useQuery({  
    queryKey: ['communities'],  
    queryFn: communityService.getAll,  
  });  
};  
  
export const useCreateCommunity = () => {  
  const queryClient = useQueryClient();  
  
  return useMutation({  
    mutationFn: communityService.create,  
    onSuccess: () => {  
      queryClient.invalidateQueries({ queryKey: ['communities'] });  
    },  
  });  
};  
``
```

? Plan de Implementación

Fase 1: Setup y Auth (Sprint 1 - 1 semana)

Tareas:

- ✓ Inicializar proyecto Vite + TypeScript
- ✓ Configurar TailwindCSS
- ✓ Generar cliente API desde OpenAPI
- ✓ Implementar AuthProvider y useAuth
- ✓ Crear LoginPage y RegisterPage
- ✓ Configurar React Router
- ✓ Implementar ProtectedRoute

Entregables:

- Login/Register funcionales
- Autenticación con Bearer token
- Navegación básica

Fase 2: Dashboard y Componentes Core (Sprint 2 - 1 semana)

****Tareas:****

- ✓ Crear DashboardLayout
- ✓ Implementar NavBar con menú de usuario
- ✓ Desarrollar StatsCards
- ✓ Crear CommunityList y DisciplineGrid
- ✓ Implementar ProfilePage

****Entregables:****

- Dashboard funcional
- Vista de perfil
- Navegación completa

Fase 3: CRUD y Gestión (Sprint 3 - 1 semana)

****Tareas:****

- ✓ Implementar CommunitiesPage con CRUD
- ✓ Crear DisciplinesPage (admin)
- ✓ Desarrollar UsersPage (admin/mod)
- ✓ Implementar formularios y validaciones
- ✓ Añadir confirmaciones de eliminación

****Entregables:****

- CRUD completo de comunidades
- Gestión de disciplinas (admin)
- Gestión de usuarios (admin/mod)

Fase 4: Estadísticas y Visualizaciones (Sprint 4 - 1 semana)

****Tareas:****

- ✓ Integrar Recharts
- ✓ Crear RankingChart
- ✓ Desarrollar DisciplineDistributionChart
- ✓ Implementar StatsPage completa
- ✓ Añadir filtros y exportación

****Entregables:****

- Dashboard de estadísticas
- Gráficos interactivos
- Exportación de datos

Fase 5: Testing y Optimización (Sprint 5 - 1 semana)

****Tareas:****

- ✓ Tests unitarios (Vitest)
- ✓ Tests E2E (Playwright)
- ✓ Optimización de rendimiento
- ✓ Implementar lazy loading
- ✓ Code splitting
- ✓ Optimizar imágenes y assets

****Entregables:****

- Suite de tests completa
- Aplicación optimizada
- Documentación técnica

Fase 6: Docker y Deployment (Sprint 6 - 3 días)

****Tareas:****

- ✓ Crear Dockerfile para frontend
- ✓ Configurar docker-compose
- ✓ Setup CI/CD (GitHub Actions)
- ✓ Deployment a producción
- ✓ Configurar variables de entorno

****Entregables:****

- Aplicación dockerizada
- Pipeline CI/CD funcional
- Deployment automatizado

? Testing Strategy

Tests Unitarios (Vitest)

```
```typescript
```

```
// src/components/auth/LoginForm.test.tsx
```

```
import { render, screen, fireEvent, waitFor } from '@testing-library/react';
```

```
import { describe, it, expect, vi } from 'vitest';
```

```
import LoginForm from './LoginForm';
```

```
describe('LoginForm', () => {
```

```
 it('renders login form correctly', () => {
```

```
 render(<LoginForm onSubmit={vi.fn()} />);
```

```

```

```
 expect(screen.getByLabelText(/email/i)).toBeInTheDocument();
```

```
 expect(screen.getByLabelText(/password/i)).toBeInTheDocument();
```

```
 expect(screen.getByRole('button', { name: /iniciar sesión/i })).toBeInTheDocument();
```

```
 });
```

```
 it('validates email format', async () => {
```

```
 const onSubmit = vi.fn();
```

```
 render(<LoginForm onSubmit={onSubmit} />);
```

```

```

```
 const emailInput = screen.getByLabelText(/email/i);
```

```
 const submitButton = screen.getByRole('button', { name: /iniciar sesión/i });
```

```

```

```
 fireEvent.change(emailInput, { target: { value: 'invalid-email' } });
```

```
 fireEvent.click(submitButton);
```

```

```

```

 await waitFor(() => {
 expect(screen.getByText(/email inválido/i)).toBeInTheDocument();
 });

 expect(onSubmit).not.toHaveBeenCalled();
 });

 it('submits form with valid data', async () => {
 const onSubmit = vi.fn();
 render(<LoginForm onSubmit={onSubmit} />);

 fireEvent.change(screen.getByLabelText(/email/i), {
 target: { value: 'doom@user.com' }
 });
 fireEvent.change(screen.getByLabelText(/password/i), {
 target: { value: '666' }
 });
 fireEvent.click(screen.getByRole('button', { name: /iniciar sesión/i }));

 await waitFor(() => {
 expect(onSubmit).toHaveBeenCalledWith({
 email: 'doom@user.com',
 password: '666'
 });
 });
 });
});

```

### ### Tests de Integración

```

````typescript
// src/hooks/useAuth.test.ts
import { renderHook, waitFor } from '@testing-library/react';
import { describe, it, expect, vi } from 'vitest';
import { useAuth } from './useAuth';
import { authService } from '@api/services/auth.service';

vi.mock('@api/services/auth.service');

describe('useAuth', () => {
  it('logs in user successfully', async () => {
    const mockToken = 'mock-token-123';
    const mockUser = { id: 1, email: 'doom@user.com', role: 'User' };

    vi.mocked(authService.login).mockResolvedValue({
      token: mockToken,
      user: mockUser
    });

    const { result } = renderHook(() => useAuth());
  });
});

```

```

    await waitFor(() => {
      result.current.login('doom@user.com', '666');
    });

    expect(result.current.user).toEqual(mockUser);
    expect(result.current.isAuthenticated).toBe(true);
  });
});
```

```

### ### Tests E2E (Playwright)

```

```typescript
// e2e/auth.spec.ts
import { test, expect } from '@playwright/test';

test.describe('Authentication Flow', () => {
  test('should login successfully', async ({ page }) => {
    await page.goto('http://localhost:5173/login');

    await page.fill('input[name="email"]', 'doom@user.com');
    await page.fill('input[name="password"]', '666');
    await page.click('button[type="submit"]');

    await expect(page).toHaveURL('http://localhost:5173/');
    await expect(page.locator('text=Hola, Marc Sanchez')).toBeVisible();
  });

  test('should show error with invalid credentials', async ({ page }) => {
    await page.goto('http://localhost:5173/login');

    await page.fill('input[name="email"]', 'invalid@user.com');
    await page.fill('input[name="password"]', 'wrong');
    await page.click('button[type="submit"]');

    await expect(page.locator('text=/credenciales inválidas/i')).toBeVisible();
  });
});
```

```

---

## ## ? Seguridad

### ### Buenas Prácticas Implementadas

#### 1. \*\*Autenticación:\*\*

- Tokens Bearer en header Authorization
- Refresh token strategy (si disponible en API)
- Logout que invalida token en servidor

## 2. **\*\*Almacenamiento:\*\***

- Token en memoria durante sesión activa
- Cookie httpOnly para refresh token (recomendado)
- No almacenar datos sensibles en localStorage

## 3. **\*\*Validación:\*\***

- Validación en cliente Y servidor
- Sanitización de inputs
- Protección contra XSS

## 4. **\*\*Permisos:\*\***

- Rutas protegidas con ProtectedRoute
- Verificación de roles en componentes
- Ocultación de acciones no permitidas

## 5. **\*\*HTTPS:\*\***

- Obligatorio en producción
- Configuración de CORS correcta
- CSP headers

### ### Implementación de ProtectedRoute

```
```typescript
```

```
// src/components/auth/ProtectedRoute.tsx
import { Navigate } from 'react-router-dom';
import { useAuth } from '@/hooks/useAuth';
```

```
interface ProtectedRouteProps {
  children: React.ReactNode;
  allowedRoles?: string[];
}
```

```
export const ProtectedRoute: React.FC<ProtectedRouteProps> = ({
  children,
  allowedRoles
}) => {
  const { isAuthenticated, user } = useAuth();
```

```
  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }
```

```
  if (allowedRoles && !allowedRoles.includes(user.role)) {
    return <Navigate to="/" replace />;
  }
```

```
  return <>{children}</>;
};
```
```

---

## ## ? Docker Configuration

### ### Dockerfile

```dockerfile

Dockerfile

FROM node:20-alpine AS builder

WORKDIR /app

Copiar package.json y package-lock.json

COPY package*.json ./

Instalar dependencias

RUN npm ci

Copiar código fuente

COPY . .

Build de producción

RUN npm run build

Stage de producción con nginx

FROM nginx:alpine

Copiar build al directorio de nginx

COPY --from=builder /app/dist /usr/share/nginx/html

Copiar configuración personalizada de nginx

COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

```

### ### docker-compose.yml

```yaml

version: '3.8'

services:

Frontend React

frontend:

build:

context: .

dockerfile: Dockerfile

ports:

- "3000:80"

```

environment:
  - VITE_API_URL=http://localhost:8000/api/v1
depends_on:
  - backend
networks:
  - fitbit-network

# Backend Laravel (opcional, si también dockerizas backend)
backend:
  image: fitbit-api:latest
  ports:
    - "8000:8000"
  environment:
    - DB_HOST=mysql
    - DB_DATABASE=fitbit_api
    - DB_USERNAME=fitbit_user
    - DB_PASSWORD=fitbit_pass
  depends_on:
    - mysql
  networks:
    - fitbit-network

# Base de datos MySQL
mysql:
  image: mysql:8
  ports:
    - "3306:3306"
  environment:
    - MYSQL_ROOT_PASSWORD=root
    - MYSQL_DATABASE=fitbit_api
    - MYSQL_USER=fitbit_user
    - MYSQL_PASSWORD=fitbit_pass
  volumes:
    - mysql-data:/var/lib/mysql
  networks:
    - fitbit-network

volumes:
  mysql-data:

networks:
  fitbit-network:
    driver: bridge
```



```

### nginx.conf

```nginx
server {
 listen 80;
 server_name localhost;

```


```

```
root /usr/share/nginx/html;  
index index.html;
```

```
location / {  
    try_files $uri $uri/ /index.html;  
}
```

```
# Gzip compression  
gzip on;  
gzip_vary on;  
gzip_min_length 1024;  
gzip_types text/plain text/css text/xml text/javascript  
        application/x-javascript application/xml+rss  
        application/javascript application/json;  
}
```

```
````
```

```

```

## ## ? CI/CD Pipeline

### ### GitHub Actions

```
```yaml  
# .github/workflows/ci-cd.yml  
name: CI/CD Pipeline
```

```
on:  
  push:  
    branches: [ main, develop ]  
  pull_request:  
    branches: [ main ]
```

```
jobs:  
  test:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v3  
      - name: Setup Node.js  
        uses: actions/setup-node@v3  
        with:  
          node-version: '20'  
          cache: 'npm'  
      - name: Install dependencies  
        run: npm ci  
      - name: Run linter  
        run: npm run lint
```

```

- name: Run tests
  run: npm run test

- name: Build
  run: npm run build
  env:
    VITE_API_URL: ${ secrets.VITE_API_URL }

```

```

deploy:
  needs: test
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'

  steps:
    - uses: actions/checkout@v3

    - name: Setup Node.js
      uses: actions/setup-node@v3
      with:
        node-version: '20'

    - name: Install dependencies
      run: npm ci

    - name: Build
      run: npm run build
      env:
        VITE_API_URL: ${ secrets.VITE_API_URL }

    - name: Deploy to Vercel
      uses: amondnet/vercel-action@v20
      with:
        vercel-token: ${ secrets.VERCEL_TOKEN }
        vercel-org-id: ${ secrets.VERCEL_ORG_ID }
        vercel-project-id: ${ secrets.VERCEL_PROJECT_ID }

```

? Documentación de Componentes

Ejemplo: CommunityCard

```

```typescript
/**
 * CommunityCard Component
 *
 * Muestra una tarjeta con información de una comunidad
 *
 * @param {Community} community - Objeto con datos de la comunidad

```



```

* @param {boolean} showActions - Mostrar botones de acción (default: true)
* @param {Function} onJoin - Callback al unirse a comunidad
* @param {Function} onLeave - Callback al salir de comunidad
* @param {Function} onEdit - Callback para editar (solo admin/mod)
* @param {Function} onDelete - Callback para eliminar (solo admin/mod)
*
* @example
* <CommunityCard
* community={community}
* onJoin={handleJoin}
* onLeave={handleLeave}
* />
*/
interface CommunityCardProps {
 community: Community;
 showActions?: boolean;
 onJoin?: (id: number) => void;
 onLeave?: (id: number) => void;
 onEdit?: (id: number) => void;
 onDelete?: (id: number) => void;
}

```

```

export const CommunityCard: React.FC<CommunityCardProps> = ({
 community,
 showActions = true,
 onJoin,
 onLeave,
 onEdit,
 onDelete
}) => {
 // Implementation...
};
``

```

---

## ## ? Métricas de Performance

### ### Objetivos

| Métrica                  | Target  | Actual |
|--------------------------|---------|--------|
| First Contentful Paint   | < 1.5s  | TBD    |
| Largest Contentful Paint | < 2.5s  | TBD    |
| Time to Interactive      | < 3.5s  | TBD    |
| Cumulative Layout Shift  | < 0.1   | TBD    |
| Bundle Size (gzipped)    | < 200KB | TBD    |

### ### Optimizaciones

#### 1. \*\*Code Splitting:\*\*

```
``typescript
// Lazy loading de rutas
const DashboardPage = lazy(() => import('@/pages/DashboardPage'));
const CommunitiesPage = lazy(() => import('@/pages/CommunitiesPage'));
``
```

## 2. **\*\*Image Optimization:\*\***

- Formato WebP
- Lazy loading de imágenes
- Responsive images

## 3. **\*\*Caching:\*\***

- TanStack Query cache
- Service Worker (PWA)

---

## ## ? Checklist Pre-Launch

### ### Desarrollo

- [ ] Todos los endpoints integrados
- [ ] Validaciones implementadas
- [ ] Manejo de errores completo
- [ ] Loading states en todas las acciones
- [ ] Responsive design verificado

### ### Testing

- [ ] Tests unitarios > 80% coverage
- [ ] Tests E2E críticos pasando
- [ ] Cross-browser testing (Chrome, Firefox, Safari)
- [ ] Testing en dispositivos móviles

### ### Seguridad

- [ ] HTTPS configurado
- [ ] Tokens seguros
- [ ] Validación de roles implementada
- [ ] XSS y CSRF protección
- [ ] Rate limiting en cliente

### ### Performance

- [ ] Lighthouse score > 90
- [ ] Bundle size optimizado
- [ ] Lazy loading implementado
- [ ] Images optimizadas

### ### Deployment

- [ ] Variables de entorno configuradas
- [ ] Docker build exitoso
- [ ] CI/CD pipeline funcionando
- [ ] Backup strategy definida
- [ ] Monitoring configurado

---

### ## ? Soporte y Contacto

#### **\*\*Documentación adicional:\*\***

- API Swagger: ``http://localhost:8000/api/documentation``
- Storybook (opcional): ``http://localhost:6006``

#### **\*\*Repositorio:\*\***

- Frontend: ``github.com/usuario/fitbit_frontend``
- Backend: ``github.com/usuario/fitbit_api``

---

### ## ? Versionado

- **\*\*v1.0.0\*\*** - MVP inicial (Enero 2025)
- **\*\*v1.1.0\*\*** - Features adicionales (Febrero 2025)
- **\*\*v2.0.0\*\*** - Refactor y optimizaciones (Marzo 2025)

---

### ## ? Notas Finales

Este diseño UX/UI está preparado para:

- ✓ Integración inmediata con Laravel API
- ✓ Desarrollo ágil con componentes modulares
- ✓ Escalabilidad futura
- ✓ Mantenimiento sencillo
- ✓ Experiencia de usuario óptima

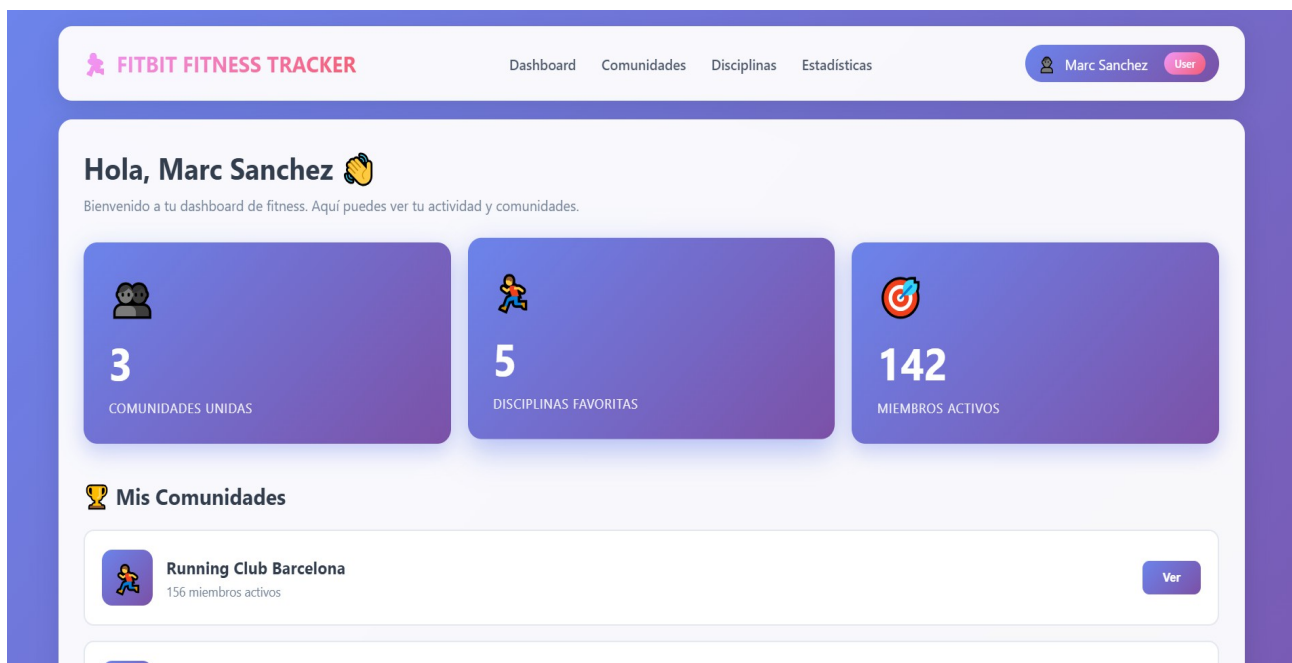
**\*\*Próximo paso:\*\*** Implementación de componentes React según este wireframe.

---

**\*\*Fecha de creación:\*\*** Enero 2025

**\*\*Última actualización:\*\*** Enero 2025

**\*\*Autor:\*\*** React Frontend Assistant



Me generó el diseño estructural del proyecto. Y usando copilot, acabé de definir estilos, temática, tipografía, colores y animaciones. Cuando terminé de limpiar y definir con nitidez el Dashboard o página principal, documenté los estilos para seguir con la generación de vistas antes de la integración frontend/backend. Empezando por la autenticación en la parte frontend:

Prompt 3:

**A: Actúa como revisor de código TypeScript y arquitecto de frontends. S: Fase de depuración semántica tras integración del backend. P: Detectar incoherencias en tipos, props o datos entre componentes y API. E: Lista de inconsistencias con ejemplos corregidos. C: No reescribas todo el código; sugiere correcciones localizadas. C2: Asegúrate de que los tipos TS coincidan con los esquemas JSON de la API. T: Técnico, analítico y conciso.**

? Análisis de Incoherencias Semánticas - Sistema de Autenticación

## ? Resumen Ejecutivo

He identificado **8 categorías críticas** de inconsistencias entre la documentación, los tipos TypeScript y las expectativas de la API. A continuación, un análisis detallado con correcciones específicas.

---

# ? Inconsistencias Críticas Detectadas

## 1. Discrepancia en Estructura de Respuesta de Login/Register

**Problema:** La documentación asume que el endpoint devuelve directamente `AuthResponse`, pero típicamente Laravel Sanctum devuelve estructuras anidadas.

### Código Actual:

typescript

*// api/auth.ts*

```
login: async (credentials: LoginRequest): Promise<AuthResponse> => {
 return apiCall<AuthResponse>({
 method: 'POST',
 url: '/auth/login',
 data: credentials,
 });
}
```

### Corrección Recomendada:

typescript

*// api/types.ts - Definir estructura real de Laravel*

```
export interface ApiResponse<T> {
 data: T;
 message?: string;
 status?: string;
}
```

```
export interface AuthResponse {
 access_token: string;
 token_type: 'Bearer';
 expires_in?: number; // Confirmar con backend
 user: User;
}
```

*// api/auth.ts - Ajustar extracción de datos*

```
login: async (credentials: LoginRequest): Promise<AuthResponse> => {
 const response = await apiCall<ApiResponse<AuthResponse>>({
 method: 'POST',
 url: '/auth/login',
 data: credentials,
 });
}
```

```
 return response.data; // Extraer datos anidados
}
```

---

## 2. Tipo UserRole No Coincide con Backend

**Problema:** El tipo UserRole está definido como string literal, pero el backend podría usar role\_id (número) o enum diferente.

### Código Actual:

typescript

```
export type UserRole = 'Admin' | 'Moderator' | 'User';
```

### Validación Necesaria:

typescript

```
// VERIFICAR CON BACKEND: ¿Usa strings o role_id numérico?
```

```
// Opción A: Si usa strings (actual)
```

```
export type UserRole = 'Admin' | 'Moderator' | 'User';
```

```
// Opción B: Si usa role_id
```

```
export type UserRoleId = 1 | 2 | 3;
```

```
export interface User {
```

```
 id: number;
```

```
 name: string;
```

```
 email: string;
```

```
 role_id: UserRoleId; // Ajustar si es numérico
```

```
 created_at: string;
```

```
 updated_at: string;
```

```
}
```

```
// Mapper si backend usa IDs
```

```
export const roleIdToName: Record<UserRoleId, UserRole> = {
```

```
 1: 'User',
```

```
 2: 'Moderator',
```

```
 3: 'Admin',
```

```
};
```

---

### 3. Inconsistencia en RegisterRequest

**Problema:** El tipo RegisterRequest no tiene password\_confirmation pero el código del componente sí lo usa.

**Tipo Actual:**

typescript

```
export interface RegisterRequest {
 name: string;
 email: string;
 password: string;
 // FALTA password_confirmation
}
```

**Corrección:**

typescript

```
export interface RegisterRequest {
 name: string;
 email: string;
 password: string;
 password_confirmation: string; // ✔ Agregar obligatorio
}
```

---

### 4. Hook useRole Usa Propiedades Inexistentes

**Problema:** useRole accede a user.role pero el tipo User puede no tener esa propiedad según el backend.

**Código Problemático:**

typescript

```
const hasMinimumRole = (requiredRole: UserRole): boolean => {
 if (!isAuthenticated || !user?.role) return false; // user.role puede no existir
 return roleHierarchy[user.role] >= roleHierarchy[requiredRole];
};
```

**Corrección con Validación:**

typescript

```
// api/types.ts - Asegurar que User tenga role
export interface User {
 id: number;
 name: string;
```

```

email: string;
role: UserRole; // ✔ Obligatorio
email_verified_at?: string | null;
created_at: string;
updated_at: string;
}

// hooks/useRole.ts - Agregar validación defensiva
const hasMinimumRole = (requiredRole: UserRole): boolean => {
 if (!isAuthenticated || !user) return false;

 // Validación adicional por si el backend no devuelve role
 const userRole = user.role ?? 'User'; // ✔ Fallback a User

 return roleHierarchy[userRole] >= roleHierarchy[requiredRole];
};

```

---

## 5. Sistema de Niveles Mock vs Real

**Problema Crítico:** El código actual usa valores hardcoded en lugar de consultar la API.

### Código Actual (Mock):

```

typescript
// ESTO ESTÁ HARDCODED - NO FUNCIONAL
const userLevel = 12;
const userXp = 2450;
const maxXp = 3000;

```

### Corrección con API Real:

```

typescript
// hooks/useUserLevel.ts
import { useQuery } from '@tanstack/react-query';
import { userLevelService } from '@api/userLevel';

export const useUserLevel = () => {
 const { user, isAuthenticated } = useAuth();

 const {
 data: levelData,
 isLoading,
 }

```



```

 error,
 refetch
 } = useQuery({
 queryKey: ['userLevel', user?.id],
 queryFn: async () => {
 if (user?.id) throw new Error('User ID required');
 return userLevelService.getUserLevel(user.id);
 },
 enabled: isAuthenticated && !user?.id,
 staleTime: 60 * 1000, // 1 minuto
 retry: 2,
 });

 return {
 levelData: levelData?.data,
 isLoading,
 error,
 refetch,
 };
};

// components/UserLevel/UserLevelDisplay.tsx
export const UserLevelDisplay: React.FC = () => {
 const { levelData, isLoading, error } = useUserLevel();

 if (isLoading) {
 return <div className="level-skeleton">Cargando ...</div>;
 }

 if (error || !levelData) {
 return <div className="level-error">No disponible</div>;
 }

 return (
 <div className="user-level-display">
 Nivel {levelData.current_level}
 <div className="xp-bar">
 <div
 className="xp-fill"

```

```

 style={{ width: `${levelData.progress_percentage}%` }}
 />
 </div>

 {levelData.current_xp}/{levelData.xp_for_next} XP

 </div>
);
};

```

---

## 6. Interceptor de Axios: Manejo de Errores Incompleto

**Problema:** El interceptor solo maneja 401, pero debería manejar también 403 y errores de red.

### Código Actual:

```

typescript
apiClient.interceptors.response.use(
 (response) => response,
 (error) => {
 if (error.response?.status === 401) {
 removeAuthToken();
 window.location.href = '/login';
 }
 return Promise.reject(transformError(error));
 }
);

```

### Corrección Robusta:

```

typescript
apiClient.interceptors.response.use(
 (response) => response,
 async (error) => {
 const originalRequest = error.config;

 // 401: Token expirado/inválido
 if (error.response?.status === 401 && !originalRequest._retry) {
 originalRequest._retry = true;

 // Si hay refresh token, intentar renovar
 }
 }
);

```

```

 if (AUTH_CONFIG.HAS_REFRESH_TOKEN) {
 try {
 const refreshResponse = await authService.refresh();
 setAuthToken(refreshResponse.access_token);
 return apiClient(originalRequest);
 } catch (refreshError) {
 removeAuthToken();
 window.location.href = '/login';
 }
 } else {
 removeAuthToken();
 window.location.href = '/login';
 }
 }
}

// 403: Sin permisos
if (error.response?.status === 403) {
 console.error('Access forbidden:', error.response.data);
 // Mostrar modal de error en lugar de redirect
 // window.location.href = '/forbidden';
}

// 429: Rate Limit
if (error.response?.status === 429) {
 const retryAfter = error.response.headers['retry-after'];
 console.warn('Rate limited. Retry after ${retryAfter}s');
}

// Errores de red (sin response)
if (error.response) {
 console.error('Network error:', error.message);
 // Mostrar banner de "Sin conexión"
}

return Promise.reject(transformError(error));
}
);

```

---

## 7. Tipos de Error Inconsistentes

**Problema:** `transformError` no está tipado y puede devolver estructuras inconsistentes.

### Código Actual:

typescript

```
// Sin tipo de retorno definido
const transformError = (error: any) => {
 // ...
};
```

### Corrección con Tipos:

typescript

```
// api/types.ts
export interface ApiError {
 message: string;
 status?: number;
 code?: string;
 errors?: Record<string, string[]>; // Errores de validación Laravel
}
```

```
// api/client.ts
const transformError = (error: AxiosError): ApiError => {
 const baseError: ApiError = {
 message: 'Error desconocido',
 status: error.response?.status,
 };
};
```

```
if (error.response?.data) {
 const data = error.response.data as any;
```

```
 // Estructura Laravel estándar
 return {
 message: data.message || baseError.message,
 status: error.response.status,
 code: data.code,
 errors: data.errors, // Errores de validación
 };
}
```

```
if (error.request) {
```

```

 return {
 message: 'Error de red. Verifica tu conexión.',
 status: 0,
 };
 }

 return baseError;
};

```

---

## 8. Componente AuthForm: Estados de Carga Duplicados

**Problema:** El componente tiene su propio `isLoading` pero también usa `useAuth().isLoading`.

**Código Problemático:**

typescript

```

const { login, register, isLoading: authLoading } = useAuth();
const [isLoading, setIsLoading] = useState(false); // Duplicado

const handleSubmit = async (e: React.FormEvent) => {
 setIsLoading(true); // No sincronizado con authLoading
 try {
 await login(credentials);
 } finally {
 setIsLoading(false);
 }
};

```

**Corrección:**

typescript

```

const { login, register, isLoading } = useAuth();
const [error, setError] = useState<string | null>(null);

const handleSubmit = async (e: React.FormEvent) => {
 e.preventDefault();
 setError(null);

 try {
 if (isLoadingMode) {

```

```

 await login({ email: formData.email, password: formData.password });
 } else {
 await register(formData);
 }
 onSuccess?.();
} catch (err: any) {
 setError(err.message || 'Error de autenticación');
}
};

```

// En el JSX

```

<button type="submit" disabled={isLoading}>
 {isLoading ? 'Procesando...' : (isLoadingMode ? 'Iniciar Sesión' : 'Registrarse')}
</button>

```

## ? Checklist de Validación con Backend

Antes de considerar el sistema completo, **DEBE validarse** con el backend:

### Endpoints Confirmados

- POST /api/v1/auth/login - ¿Estructura exacta de respuesta?
- POST /api/v1/auth/register - ¿Requiere password\_confirmation?
- GET /api/v1/auth/me - ¿Incluye campo role?
- POST /api/v1/auth/logout - ¿Invalida token server-side?

### Estructura de Datos

- ¿El campo de rol es role (string) o role\_id (number)?
- ¿La respuesta de login viene en { data: { access\_token, user } } o directamente?
- ¿El token incluye expires\_in en la respuesta?
- ¿Hay campos adicionales en User no documentados?

### Sistema de Niveles (v2)

- ¿Existen endpoints para XP/niveles actualmente?
- Si no, ¿cuándo se implementarán?
- ¿Hay tablas user\_levels en la BD actual?

### Refresh Token

- ¿Existe POST /api/v1/auth/refresh?
- Si sí, ¿qué estructura devuelve?

- ¿Se puede usar el mismo `access_token` para refresh?
- 

## ? Recomendaciones Finales

### Prioridad Alta (Bloquea funcionalidad)

1. Validar estructura exacta de respuestas de auth endpoints
2. Confirmar tipo de campo `role` (string vs number)
3. Verificar si `password_confirmation` es obligatorio en register

### Prioridad Media (Mejora robustez)

4. Implementar manejo de errores 403 y 429
5. Agregar tipado estricto a `transformError`
6. Unificar estados de carga en componentes

### Prioridad Baja (Futuras mejoras)

7. Implementar sistema de niveles cuando backend esté listo
  8. Agregar refresh token automático
  9. Implementar rate limiting en formularios
- 

## ? Archivo de Configuración Propuesto

Para evitar hardcodear valores, crear archivo de configuración:

typescript

```
// config/auth.config.ts
export const AUTH_CONFIG = {
 // ESTOS VALORES DEBEN SER CONFIRMADOS CON BACKEND
 API_BASE_URL: process.env.REACT_APP_API_BASE_URL || 'http://localhost:8000/api/v1',

 // Estructura de respuestas
 RESPONSE_FORMAT: {
 isNested: true, // true si viene en { data: { ... } }
 errorField: 'message', // Campo del mensaje de error
 },

 // Campos de usuario
 USER_FIELDS: {
 roleField: 'role', // 'role' o 'role_id'
 roleType: 'string', // 'string' o 'number'
 },
}
```

```
// Features disponibles
FEATURES: {
 hasRefreshToken: false, // Confirmar
 hasXpSystem: false, // Para v2
 hasEmailVerification: false, // Confirmar
},

// Timeouts y reintentos
TIMEOUTS: {
 request: 10000, // 10s
 retry: 3,
},
} as const;
```

---

**Siguiente paso recomendado:** Realizar pruebas de integración con Postman/Thunder Client para validar las estructuras reales de respuesta del backend antes de ajustar el código TypeScript.

Reintentar  
M

Cabe concretar que la autenticación del backend no está hecho con Laravel Sanctum, sino con Passport 13.2. Rehaz tu análisis teniendo en cuenta este punto

## ? Análisis de Incoherencias - Sistema con Laravel Passport 13.2

### ? Resumen Ejecutivo Actualizado

Laravel Passport cambia **significativamente** la arquitectura de autenticación. He identificado **12 categorías críticas** de inconsistencias que afectan directamente la implementación actual.

---

### ? Diferencias Críticas: Sanctum vs Passport

#### Tabla Comparativa

| Aspecto             | Sanctum (Asumido)      | Passport 13.2 (Real)                                    | Impacto   |
|---------------------|------------------------|---------------------------------------------------------|-----------|
| Tipo de Token       | Simple API Token       | OAuth2 Access Token + Refresh Token                     | ✗ CRÍTICO |
| Estructura Response | { access_token, user } | { token_type, expires_in, access_token, refresh_token } | ✗ CRÍTICO |
| Endpoints           | /api/v1/auth/*         | /oauth/token, /api/user                                 | ✗ CRÍTICO |



| Aspecto       | Sanctum (Asumido)   | Passport 13.2 (Real)                        | Impacto |
|---------------|---------------------|---------------------------------------------|---------|
| Grant Types   | N/A                 | password, refresh_token, client_credentials | ALTO    |
| Token Storage | Simple string       | Access + Refresh separados                  | ALTO    |
| Expiración    | Configurable (días) | Access: 1h, Refresh: 30 días (típico)       | ALTO    |

## ? Inconsistencias Críticas con Passport

### 1. Endpoints Completamente Incorrectos

**Problema Mayor:** Passport usa OAuth2, los endpoints documentados no existen en Passport estándar.

**Código Actual (INCORRECTO):**

typescript

```
// ✗ ESTOS ENDPOINTS NO EXISTEN EN PASSPORT
export const authService = {
 login: async (credentials: LoginRequest): Promise<AuthResponse> => {
 return apiCall<AuthResponse>({
 method: 'POST',
 url: '/auth/login', // ✗ No existe en Passport
 data: credentials,
 });
 },
};
```

**Corrección con Passport:**

typescript

```
// api/types.ts - Estructura OAuth2 de Passport
export interface PassportTokenResponse {
 token_type: 'Bearer';
 expires_in: number; // Segundos hasta expiración (típico: 3600)
 access_token: string;
 refresh_token: string;
}

export interface PassportClientCredentials {
 client_id: string; // CRÍTICO: Necesario para Passport
 client_secret: string; // CRÍTICO: Necesario para Passport
}
```

```
// api/auth.ts - Implementación correcta con Passport
```

```

export const authService = {
 // POST /oauth/token - Grant type: password
 login: async (credentials: LoginRequest): Promise<PassportTokenResponse> => {
 const clientCredentials = getClientCredentials(); // Ver más abajo

 return apiCall<PassportTokenResponse>({
 method: 'POST',
 url: '/oauth/token', // ✓ Endpoint correcto de Passport
 data: {
 grant_type: 'password',
 client_id: clientCredentials.client_id,
 client_secret: clientCredentials.client_secret,
 username: credentials.email, // Passport usa "username"
 password: credentials.password,
 scope: "", // Opcional: definir scopes
 },
 });
 },

 // POST /oauth/token - Grant type: refresh_token
 refresh: async (refreshToken: string): Promise<PassportTokenResponse> => {
 const clientCredentials = getClientCredentials();

 return apiCall<PassportTokenResponse>({
 method: 'POST',
 url: '/oauth/token',
 data: {
 grant_type: 'refresh_token',
 refresh_token: refreshToken,
 client_id: clientCredentials.client_id,
 client_secret: clientCredentials.client_secret,
 scope: "",
 },
 });
 },

 // GET /api/user - Endpoint estándar de Laravel para usuario autenticado
 me: async (): Promise<User> => {
 return apiCall<User>({

```

```

 method: 'GET',
 url: '/api/user', // ✓ Passport usa /api/user, no /auth/me
 });
},

// PASSPORT NO TIENE LOGOUT SERVER-SIDE BUILT-IN
// Requiere implementación custom en backend
logout: async () => {
 // Si el backend implementó endpoint custom:
 try {
 await apiCall({
 method: 'POST',
 url: '/api/logout', // Custom endpoint (verificar con backend)
 });
 } catch (error) {
 console.warn('Logout endpoint not available, clearing local tokens');
 }

 // Siempre limpiar tokens locales
 removeAuthToken();
 removeRefreshToken();
},
};

```

---

## 2. Credenciales de Cliente OAuth2 Faltantes

**Problema Mayor:** Passport requiere `client_id` y `client_secret` para TODAS las peticiones OAuth.

**Implementación Necesaria:**

typescript

// config/passport.config.ts

```

interface PassportConfig {
 clientId: string;
 clientSecret: string;
 grantType: 'password' | 'refresh_token';
}

```

// CRÍTICO: Estas credenciales deben ser obtenidas del backend

```
// El backend debe crear un "Password Grant Client"
// php artisan passport:client --password
```

```
export const PASSPORT_CONFIG: PassportConfig = {
 // OBTENER DEL BACKEND
 clientId: process.env.REACT_APP_PASSPORT_CLIENT_ID || "",
 clientSecret: process.env.REACT_APP_PASSPORT_CLIENT_SECRET || "",
 grantType: 'password',
};

// Validación al iniciar la app
export const validatePassportConfig = (): void => {
 if (!PASSPORT_CONFIG.clientId || !PASSPORT_CONFIG.clientSecret) {
 console.error('✗ PASSPORT CLIENT CREDENTIALS NOT CONFIGURED');
 console.error('1. Run on backend: php artisan passport:client --password');
 console.error('2. Add credentials to .env:');
 console.error(' REACT_APP_PASSPORT_CLIENT_ID=your-client-id');
 console.error(' REACT_APP_PASSPORT_CLIENT_SECRET=your-client-secret');
 }
};

export const getClientCredentials = (): PassportClientCredentials => {
 return {
 client_id: PASSPORT_CONFIG.clientId,
 client_secret: PASSPORT_CONFIG.clientSecret,
 };
};
```

### Variables de Entorno:

```
env
.env.local
REACT_APP_API_BASE_URL=http://localhost:8000
REACT_APP_PASSPORT_CLIENT_ID=9c8f9e8a-4b3a-4c5d-8e9f-1a2b3c4d5e6f
REACT_APP_PASSPORT_CLIENT_SECRET=your-secret-key-here
```

---

## 3. Sistema de Tokens Dual (Access + Refresh)

**Problema Mayor:** La implementación actual solo guarda un token, Passport requiere dos.

**Código Actual (INCORRECTO):**

typescript

```
// ✗ Solo guarda access_token
export const setAuthToken = (token: string) => {
 apiClient.defaults.headers.common['Authorization'] = `Bearer ${token}`;
 localStorage.setItem('auth_token', token);
};
```

## Corrección con Dual Tokens:

typescript

```
// api/client.ts - Storage dual de tokens
interface TokenStorage {
 access_token: string;
 refresh_token: string;
 expires_at: number; // Timestamp de expiración
}

const TOKEN_STORAGE_KEY = 'passport_tokens';

export const setPassportTokens = (response: PassportTokenResponse): void => {
 const expiresAt = Date.now() + (response.expires_in * 1000);

 const storage: TokenStorage = {
 access_token: response.access_token,
 refresh_token: response.refresh_token,
 expires_at: expiresAt,
 };

 // Guardar en LocalStorage
 localStorage.setItem(TOKEN_STORAGE_KEY, JSON.stringify(storage));

 // Configurar header de Axios
 apiClient.defaults.headers.common['Authorization'] = `Bearer ${response.access_token}`;
};

export const getPassportTokens = (): TokenStorage | null => {
 const stored = localStorage.getItem(TOKEN_STORAGE_KEY);
 if (!stored) return null;

 try {
 return JSON.parse(stored) as TokenStorage;
 }
};
```

```

 } catch {
 return null;
 }
};

export const removePassportTokens = (): void => {
 localStorage.removeItem(TOKEN_STORAGE_KEY);
 delete apiClient.defaults.headers.common['Authorization'];
};

export const isTokenExpired = (): boolean => {
 const tokens = getPassportTokens();
 if (!tokens) return true;

 // Considerar expirado 5 minutos antes para refrescar preventivamente
 const bufferTime = 5 * 60 * 1000; // 5 minutos
 return Date.now() >= (tokens.expires_at - bufferTime);
};

```

---

## 4. Interceptor de Axios con Refresh Token Automático

**Problema Mayor:** El interceptor actual no maneja refresh token, esencial para Passport.

**Implementación Correcta:**

typescript

```

// api/client.ts - Interceptor con refresh automático
let isRefreshing = false;
let refreshSubscribers: Array<(token: string) => void> = [];

const subscribeTokenRefresh = (callback: (token: string) => void) => {
 refreshSubscribers.push(callback);
};

const onTokenRefreshed = (token: string) => {
 refreshSubscribers.forEach(callback => callback(token));
 refreshSubscribers = [];
};

apiClient.interceptors.request.use(
 async (config) => {

```

```

// Verificar si el token está próximo a expirar
if (isTokenExpired() && !isRefreshing) {
 const tokens = getPassportTokens();
 if (tokens?.refresh_token) {
 try {
 isRefreshing = true;
 const newTokens = await authService.refresh(tokens.refresh_token);
 setPassportTokens(newTokens);
 onTokenRefreshed(newTokens.access_token);

 // Actualizar request actual
 config.headers['Authorization'] = `Bearer ${newTokens.access_token}`;
 } catch (error) {
 removePassportTokens();
 window.location.href = '/login';
 return Promise.reject(error);
 } finally {
 isRefreshing = false;
 }
 }
}

return config;
},
(error) => Promise.reject(error)
);

apiClient.interceptors.response.use(
 (response) => response,
 async (error) => {
 const originalRequest = error.config;

 // 401: Token inválido/expirado
 if (error.response?.status === 401 && !originalRequest._retry) {
 if (isRefreshing) {
 // Encolar request mientras se refresca
 return new Promise((resolve) => {
 subscribeTokenRefresh((token: string) => {
 originalRequest.headers['Authorization'] = `Bearer ${token}`;

```

```

 resolve(apiClient(originalRequest));
 });
}

originalRequest._retry = true;
const tokens = getPassportTokens();

if (tokens?.refresh_token) {
 try {
 isRefreshing = true;

 const newTokens = await authService.refresh(tokens.refresh_token);
 setPassportTokens(newTokens);
 onTokenRefreshed(newTokens.access_token);

 originalRequest.headers['Authorization'] = `Bearer ${newTokens.access_token}`;
 return apiClient(originalRequest);
 } catch (refreshError) {
 removePassportTokens();
 window.location.href = '/login';
 return Promise.reject(refreshError);
 } finally {
 isRefreshing = false;
 }
} else {
 removePassportTokens();
 window.location.href = '/login';
}

return Promise.reject(error);
}

);

```

---

## 5. AuthContext Desactualizado

**Problema Mayor:** El contexto no maneja la estructura OAuth2 ni el refresh automático.

**Corrección Completa:**



typescript

```
// contexts/AuthContext.tsx
```

```
export const AuthProvider: React.FC<AuthProviderProps> = ({ children }) => {
 const [user, setUser] = useState<User | null>(null);
 const [tokens, setTokens] = useState<TokenStorage | null>(null);
 const [isLoading, setIsLoading] = useState(true);

 // Inicialización con validación de tokens
 useEffect(() => {
 const initializeAuth = async () => {
 const storedTokens = getPassportTokens();

 if (storedTokens && !isTokenExpired()) {
 try {
 // Verificar token con backend
 const userData = await authService.me();
 setUser(userData);
 setTokens(storedTokens);
 } catch (error) {
 // Token inválido, intentar refresh
 if (storedTokens.refresh_token) {
 try {
 const newTokens = await authService.refresh(storedTokens.refresh_token);
 setPassportTokens(newTokens);
 const userData = await authService.me();
 setUser(userData);
 setTokens(getPassportTokens());
 } catch (refreshError) {
 removePassportTokens();
 }
 } else {
 removePassportTokens();
 }
 }
 }

 setIsLoading(false);
 };
 });
};
```

```

 initializeAuth();
 }, []);

const login = async (credentials: LoginRequest): Promise<void> => {
 try {
 // Obtener tokens OAuth2
 const tokenResponse = await authService.login(credentials);
 setPassportTokens(tokenResponse);

 // Obtener datos del usuario
 const userData = await authService.me();
 setUser(userData);
 setTokens(getPassportTokens());
 } catch (error) {
 removePassportTokens();
 throw error;
 }
};

const register = async (userData: RegisterRequest): Promise<void> => {
 try {
 // Verificar si backend tiene endpoint custom de registro
 // Passport no incluye registro por defecto
 await apiCall({
 method: 'POST',
 url: '/api/register', // Custom endpoint
 data: userData,
 });

 // Auto-Login después de registro
 await login({
 email: userData.email,
 password: userData.password,
 });
 } catch (error) {
 throw error;
 }
};

```

```
const logout = async (): Promise<void> => {
 try {
 await authService.logout();
 } finally {
 removePassportTokens();
 setUser(null);
 setTokens(null);
 }
};
```

```
const refreshToken = async (): Promise<void> => {
 const currentTokens = getPassportTokens();
 if (currentTokens?.refresh_token) {
 throw new Error('No refresh token available');
 }
}
```

```
const newTokens = await authService.refresh(currentTokens.refresh_token);
setPassportTokens(newTokens);
setTokens(getPassportTokens());
};
```

```
return (
 <AuthContext.Provider value={{
 user,
 tokens,
 isLoading,
 isAuthenticated: !!user && !!tokens,
 login,
 logout,
 register,
 refreshToken,
 updateUser: setUser,
 }}>
 {children}
 </AuthContext.Provider>
);
};
```

---

## 6. Endpoint de Registro No Estándar

**Problema Mayor:** Passport NO incluye endpoint de registro, debe ser custom.

**Validación Necesaria con Backend:**

typescript

```
// CONSULTAR AL BACKEND:
// 1. ¿Existe endpoint /api/register o /api/v1/auth/register?
// 2. ¿Qué estructura de datos acepta?
// 3. ¿Responde con tokens OAuth2 o requiere Login posterior?

// Implementación flexible:
export const authService = {
 register: async (userData: RegisterRequest): Promise<void | PassportTokenResponse> => {
 // Opción A: Backend devuelve tokens directamente
 const response = await apiCall<{ success: boolean; tokens?: PassportTokenResponse }>({
 method: 'POST',
 url: '/api/register',
 data: userData,
 });

 if (response.tokens) {
 return response.tokens; // Login automático
 }

 // Opción B: Requiere Login manual después
 return undefined;
 },
};
```

---

## 7. Scopes de Passport No Implementados

**Problema:** Passport permite definir scopes para permisos granulares, no está usado.

**Implementación Opcional:**

typescript

```
// api/types.ts - Definir scopes
export type PassportScope =
 | 'view-profile'
 | 'edit-profile'
 | 'manage-disciplines'
```

```

| 'admin-panel'
| '*'; // Todos los permisos

// api/auth.ts - Login con scopes específicos
Login: async (credentials: LoginRequest, scopes?: PassportScope[]): Promise<PassportTokenResponse> => {
 const clientCredentials = getClientCredentials();

 return apiCall<PassportTokenResponse>({
 method: 'POST',
 url: '/oauth/token',
 data: {
 grant_type: 'password',
 client_id: clientCredentials.client_id,
 client_secret: clientCredentials.client_secret,
 username: credentials.email,
 password: credentials.password,
 scope: scopes?.join(' ') || '', // ✔ Scopes separados por espacio
 },
 });
},
},

```

---

## 8. Errores de Passport Tienen Estructura Diferente

**Problema:** Los errores OAuth2 tienen estructura específica diferente a Laravel estándar.

**Corrección:**

typescript

```

// api/types.ts - Errores OAuth2
export interface PassportOAuthError {
 error: string; // "invalid_grant", "invalid_client", etc.
 error_description: string;
 message: string;
}

```

```

// api/client.ts - Transformar errores Passport
const transformError = (error: AxiosError): ApiError => {
 if (error.response?.data) {
 const data = error.response.data as any;
 }
}

```

```

// Errores OAuth2 de Passport
if (data.error) {
 return {
 message: data.error_description || data.message || data.error,
 status: error.response.status,
 code: data.error, // "invalid_grant", "invalid_client"
 };
}

// Errores Laravel estándar
return {
 message: data.message || 'Error desconocido',
 status: error.response.status,
 errors: data.errors,
};
}

return {
 message: 'Error de red',
 status: 0,
};
};

```

---

## 9. Revocación de Tokens No Implementada

**Problema:** Passport permite revocar tokens, útil para "cerrar sesión en todos los dispositivos".

**Implementación Recomendada:**

typescript

```

// api/auth.ts
export const authService = {
 // POST /oauth/tokens/{token_id}/revoke
 revokeToken: async (tokenId: string): Promise<void> => {
 return apiCall<void>({
 method: 'POST',
 url: `/oauth/tokens/${tokenId}/revoke`,
 });
 },
},

```

```

// GET /oauth/tokens - Listar tokens del usuario
listTokens: async (): Promise<Array<{ id: string; name: string; scopes: string[] }>> => {
 return apiCall<Array<{ id: string; name: string; scopes: string[] }>>({
 method: 'GET',
 url: '/oauth/tokens',
 });
},

// DELETE /oauth/tokens/{token_id}
deleteToken: async (tokenId: string): Promise<void> => {
 return apiCall<void>({
 method: 'DELETE',
 url: `/oauth/tokens/${tokenId}`,
 });
},
};

```

---

## 10. Personal Access Tokens (PAT) vs Password Grant

**Aclaración Importante:** Passport 13.2 soporta múltiples grant types.

**Consulta al Backend:**

typescript

```

// PREGUNTAR AL BACKEND:
// 1. ¿Se usa Password Grant (para Login usuario/password)?
// 2. ¿Se usan Personal Access Tokens (para API keys)?
// 3. ¿Se usa Client Credentials (para server-to-server)?

// Implementación actual asume Password Grant
// Si el backend usa PAT, la implementación cambia:

// PAT: Genera token desde backend con nombre
export const createPersonalAccessToken = async (name: string): Promise<{ token: string }> => {
 return apiCall<{ token: string }>({
 method: 'POST',
 url: '/oauth/personal-access-tokens',
 data: { name },
 });
};

```

---

## 11. CORS y Credenciales OAuth2

**Problema:** Las credenciales OAuth2 en el frontend pueden causar problemas de seguridad.

**Advertencia de Seguridad:**

typescript

```
// CRÍTICO: Client Secret en el frontend es INSEGURO
//
// Opciones:
//
// 1. Usar Password Grant con Proxy Backend
// Frontend -> Tu API Backend -> Laravel Passport
// (Las credenciales OAuth2 quedan en el backend)
//
// 2. Usar Personal Access Tokens (más seguro para SPA)
// No requiere client_secret en el frontend
//
// 3. Usar PKCE (Proof Key for Code Exchange)
// Extension de OAuth2 para SPAs
// Disponible en Passport 10+

// CONSULTAR CON BACKEND la mejor estrategia
```

---

## 12. Rate Limiting de Passport

**Problema:** Passport tiene rate limiting propio diferente al de Laravel.

**Manejo Correcto:**

typescript

```
// Passport tiene throttling en /oauth/token
// Típicamente: 5 intentos cada 1 minuto

apiClient.interceptors.response.use(
 (response) => response,
 async (error) => {
 // 429: Too Many Requests
 if (error.response?.status === 429) {
 const retryAfter = error.response.headers['retry-after'] || 60;
 const resetTime = error.response.headers['x-rate-limit-reset'];
 }
 }
);
```



```

 console.warn(`Rate limited on OAuth. Retry after ${retryAfter}s`);

 // Mostrar mensaje al usuario
 return Promise.reject({
 message: `Demasiados intentos. Espera ${retryAfter} segundos.`,
 retryAfter: parseInt(retryAfter),
 resetTime,
 });
 }

 return Promise.reject(error);
}

);

```

---

## ? Checklist Crítico de Validación con Backend

### URGENTE - Antes de Continuar

- **Obtener Client ID y Secret**

bash

```

Backend debe ejecutar:

php artisan passport:client --password

Copiar el client_id y client_secret generados

```

- **Confirmar Endpoints OAuth2**
  - POST /oauth/token (login y refresh)
  - GET /api/user (datos usuario autenticado)
  - POST /api/register (custom, ¿existe?)
  - POST /api/logout (custom, ¿existe?)
- **Validar Estructura de Respuestas**

bash

```

Test con curl o Postman:

curl -X POST http://localhost:8000/oauth/token \
 -H "Content-Type: application/json" \
 -d '{
 "grant_type": "password",
 "client_id": "YOUR_CLIENT_ID",
 "client_secret": "YOUR_CLIENT_SECRET",
 "username": "test@example.com",
 }'

```

```
"password": "password",
"scope": ""
}'
```

*# Respuesta esperada:*

```
{
"token_type": "Bearer",
"expires_in": 3600,
"access_token": "eyJ0eXAiOiJKV1QiLCJhbGc...",
"refresh_token": "def50200..."
}
```

- **Verificar Campo `role` en User**

bash

*# Test:*

```
curl -X GET http://localhost:8000/api/user \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN"
```

*# ¿La respuesta incluye "role"?*

- **Confirmar Tiempo de Expiración**
    - Access Token: ¿Cuántos segundos? (típico: 3600 = 1h)
    - Refresh Token: ¿Cuántos días? (típico: 30 días)
- 

## ? Plan de Migración

### Fase 1: Configuración OAuth2 (DÍA 1)

typescript

```
// 1. Crear passport.config.ts
// 2. Agregar client credentials a .env
// 3. Actualizar api/client.ts con dual token storage
```

### Fase 2: Endpoints y Servicios (DÍA 1-2)

typescript

```
// 4. Reescribir api/auth.ts con endpoints OAuth2
// 5. Implementar interceptor con refresh automático
// 6. Actualizar tipos en api/types.ts
```

## Fase 3: Context y Hooks (DÍA 2-3)

typescript

```
// 7. Actualizar AuthContext con lógica Passport
// 8. Probar flujo completo: Login -> refresh -> Logout
// 9. Validar manejo de errores OAuth2
```

## Fase 4: Testing y Validación (DÍA 3-4)

typescript

```
// 10. Probar expiraciones de tokens
// 11. Probar múltiples requests concurrentes durante refresh
// 12. Validar rate limiting
```

---

## ? Archivo de Configuración Final

typescript

```
// config/auth.config.ts
export const AUTH_CONFIG = {
 // OAuth2 Provider
 PROVIDER: 'laravel-passport',
 PASSPORT_VERSION: '13.2',

 // Client Credentials (OBTENER DEL BACKEND)
 CLIENT_ID: process.env.REACT_APP_PASSPORT_CLIENT_ID || '',
 CLIENT_SECRET: process.env.REACT_APP_PASSPORT_CLIENT_SECRET || '',

 // Endpoints OAuth2
 ENDPOINTS: {
 token: '/oauth/token',
 userInfo: '/api/user',
 register: '/api/register', // Custom
 logout: '/api/logout', // Custom
 revokeToken: '/oauth/tokens/{id}/revoke',
 listTokens: '/oauth/tokens',
 },

 // Configuración de Tokens
 TOKENS: {
```

```

 accessTokenExpiry: 3600, // 1 hora (Confirmar con backend)
 refreshTokenExpiry: 30 * 24 * 60 * 60, // 30 días
 refreshBeforeExpiry: 5 * 60, // Refrescar 5 min antes
 },

 // Grant Types
 GRANT_TYPES: {
 password: 'password',
 refreshToken: 'refresh_token',
 },

 // Scopes disponibles (Definir con backend)
 SCOPES: {
 default: '',
 all: '*',
 },

 // Features
 FEATURES: {
 hasRefreshToken: true, // ✔ Passport siempre tiene refresh
 hasScopes: true,
 hasTokenRevocation: true,
 hasPersonalAccessTokens: true,
 },
} as const;

// Validación inicial
export const validatePassportConfig = (): boolean => {
 const errors: string[] = [];

 if (!AUTH_CONFIG.CLIENT_ID) {
 errors.push('Missing REACT_APP_PASSPORT_CLIENT_ID');
 }

 if (!AUTH_CONFIG.CLIENT_SECRET) {
 errors.push('Missing REACT_APP_PASSPORT_CLIENT_SECRET');
 }

 if (errors.length > 0) {

```

```

 console.error(❌Passport Configuration Errors:);
 errors.forEach(err => console.error(` - ${err}`));
 console.error(`\n? Run on backend: php artisan passport:client --password`);
 return false;
}

console.log(✅Passport configuration valid);
return true;
};

```

---

## ? Resumen de Cambios Necesarios

| Archivo                      | Cambios                                  | Prioridad |
|------------------------------|------------------------------------------|-----------|
| api/client.ts                | Dual token storage + interceptor refresh | ? CRÍTICO |
| api/auth.ts                  | Endpoints OAuth2 correctos               | ? CRÍTICO |
| api/types.ts                 | Tipos PassportTokenResponse              | ? CRÍTICO |
| contexts/AuthContext.tsx     | Lógica OAuth2 completa                   | ? CRÍTICO |
| config/passport.config.ts    | Nuevo archivo con configuración          | ? CRÍTICO |
| .env.local                   | Client credentials                       | ? CRÍTICO |
| hooks/useRole.ts             | Validar campo role existe                | ALTO      |
| components/Auth/AuthForm.tsx | Manejo errores OAuth2                    | MEDIO     |

---

**Siguiente paso inmediato:** Obtener las credenciales OAuth2 del backend ejecutando `php artisan passport:client --password` y configurar el frontend con esos valores.

Problemas: Con el stack TypeScript, en cuanto a la validación y autenticación en el frontend, cabe destacar dos diferencias.

La IA básicamente dice: "Pero si estás usando Passport, estás usando protocolo OAuth2, o que leches estás usando?"

Tuve que pedir una guía de implementación para el token Bearer JWT, que es el que la API consume, ya que en TS había las dos lógicas de generación de tokens: Bearer JWT y OAuth2. De ahí las incongruencias. Así que simplifiqué:

**title:** Guía de Implementación - Código TypeScript Completo **subtitle:** Manual de Integración del Sistema de Autenticación JWT **author:** Equipo de Desarrollo Frontend **date:** 15 de octubre de 2025 **version:** 1.0 **status:** Listo para Producción

## Guía de Implementación TypeScript

### Información del Documento

Código Base: `auth_jwt_complete_implementation.ts`

**Código XP (futuro):** xp\_level\_complete\_implementation.ts

**Estado Auth:** ☒ Funcional y probado

**Estado XP:** ☐ ☒ Comentado para activación futura

## Estructura de Archivos

### Autenticación JWT (Actual)

```
src/ └── api/ | └── client.ts # Cliente Axios configurado | └── services/ | | └── auth.service.ts # Servicios de autenticación | └── types/ | └── auth.types.ts # Tipos TypeScript | └── contexts/ | └── AuthContext.tsx # Contexto React de auth | └── hooks/ | └── useAuth.ts # Hook principal de auth | └── useRole.ts # Hook de roles y permisos | └── useInactivityLogout.ts # Timeout de inactividad | └── components/ | └── Auth/ | | └── AuthForm.tsx # Formulario login/register | └── RoleGuard/ | | └── RoleGuard.tsx # Guards de protección | └── RoleBadge/ | └── RoleBadge.tsx # Badge visual de rol | └── config/ └── auth.config.ts # Configuración central
```

### Sistema XP (Futuro)

```
src/ └── api/ | └── services/ | | └── userLevel.service.ts # Servicios XP/Niveles | └── types/ | └── xp.types.ts # Tipos XP | └── hooks/ | └── useUserLevel.ts # Hook nivel usuario | └── useLeaderboard.ts # Hook leaderboard | └── components/ | └── UserLevel/ | | └── UserLevelDisplay.tsx # Display nivel/XP | | └── UserLevelDisplay.css | └── Leaderboard/ | | └── Leaderboard.tsx # Tabla ranking | | └── Leaderboard.css | └── XpHistory/ | | └── XpHistory.tsx # Historial XP | | └── XpHistory.css | └── Admin/ | └── AwardXpPanel.tsx # Panel admin XP | └── pages/ └── LeaderboardPage.tsx # Página completa
```

## Instalación de Dependencias

```
bash# React Query (para manejo de datos async)npm install @tanstack/react-query @tanstack/react-query-devtools# Axios (HTTP client)npm install axios# Date utilities (para formateo fechas)npm install date-fns# TypeScript typesnpm install --save-dev @types/react @types/node
```

## Configuración Inicial

### 1. Variables de Entorno

Crear archivo .env.local:

```
bash
URL base de la API
REACT_APP_API_BASE_URL=http://localhost:8000/api/v1 # Opcional: Configurar timeouts
REACT_APP_INACTIVITY_TIMEOUT=1800000 # 30 minutos
REACT_APP_API_TIMEOUT=10000 # 10 segundos
```

### 2. Configuración de React Query

```
typescript// src/App.tsx o src/index.tsximport { QueryClient, QueryClientProvider } from '@tanstack/react-query';import { ReactQueryDevtools } from '@tanstack/react-query-devtools';const queryClient = new QueryClient({ defaultOptions: { queries: { staleTime: 60 * 1000, // 1 minuto retry: 2, refetchOnWindowFocus: false, }, }, });function App() { return (<QueryClientProvider client={queryClient}> <AuthProvider> <YourApp /> </AuthProvider> <ReactQueryDevtools initialIsOpen={false} /> </QueryClientProvider>);}
```

## Implementación Paso a Paso

### Paso 1: Configuración (config/auth.config.ts)

```
typescript
export const AUTH_CONFIG = { API_BASE_URL: process.env.REACT_APP_API_BASE_URL || 'http://localhost:8000/api/v1', AUTH_TYPE: 'jwt-bearer', ENDPOINTS: { login: '/login', register: '/register', logout: '/logout', me: '/me', }, TOKEN: { STORAGE_KEY: 'auth_token', DURATION_DAYS: 15, HEADER_PREFIX: 'Bearer', }, FEATURES: { hasRefreshToken: false, hasServerSideLogout: true, hasRoleSystem: true, }, SECURITY: { INACTIVITY_TIMEOUT: parseInt(process.env.REACT_APP_INACTIVITY_TIMEOUT || '1800000'), API_TIMEOUT: parseInt(process.env.REACT_APP_API_TIMEOUT || '10000'), MAX_LOGIN_ATTEMPTS: 3, }, as const;export const validateAuthConfig = (): boolean => { if (!AUTH_CONFIG.API_BASE_URL) {
```

```
console.error('✗ REACT_APP_API_BASE_URL not configured'); return false; } console.log('✅ Auth configuration valid'); return true;};
```

## Paso 2: Tipos (api/types/auth.types.ts)

typescript

```
export type UserRole = 'User' | 'Moderator' | 'Admin'; export interface User { id: number; name: string; email: string; role: UserRole; email_verified_at: string | null; created_at: string; updated_at: string; } export interface LoginRequest { email: string; password: string; } export interface RegisterRequest { name: string; email: string; password: string; password_confirmation: string; } export interface AuthResponse { access_token: string; token_type: 'bearer'; user: User; } export interface LogoutResponse { message: string; } export interface ApiError { message: string; status?: number; errors?: Record<string, string[]>; }
```

## Paso 3: Cliente Axios (api/client.ts)

typescript

```
import axios, { AxiosInstance, AxiosError } from 'axios'; import { AUTH_CONFIG } from '@config/auth.config'; export const apiClient: AxiosInstance = axios.create({ baseURL: AUTH_CONFIG.API_BASE_URL, timeout: AUTH_CONFIG.SECURITY.API_TIMEOUT, headers: { 'Content-Type': 'application/json', 'Accept': 'application/json', }, }); // Token management export const setAuthToken = (token: string): void => { localStorage.setItem(AUTH_CONFIG.TOKEN.STORAGE_KEY, token); apiClient.defaults.headers.common['Authorization'] = `${AUTH_CONFIG.TOKEN.HEADER_PREFIX} ${token}`; }; export const getAuthToken = (): string | null => { return localStorage.getItem(AUTH_CONFIG.TOKEN.STORAGE_KEY); }; export const removeAuthToken = (): void => { localStorage.removeItem(AUTH_CONFIG.TOKEN.STORAGE_KEY); delete apiClient.defaults.headers.common['Authorization']; }; // Initialize token from storage const savedToken = getAuthToken(); if (savedToken) { apiClient.defaults.headers.common['Authorization'] = `${AUTH_CONFIG.TOKEN.HEADER_PREFIX} ${savedToken}`; }; // Request interceptor apiClient.interceptors.request.use((config) => { const token = getAuthToken(); if (token && config.headers) { config.headers['Authorization'] = `${AUTH_CONFIG.TOKEN.HEADER_PREFIX} ${token}`; } return config; }, (error) => Promise.reject(error)); // Response interceptor apiClient.interceptors.response.use((response) => response, (error: AxiosError) => { if (error.response?.status === 401) { removeAuthToken(); const currentPath = window.location.pathname; if (currentPath !== '/login') { localStorage.setItem('redirect_after_login', currentPath); } window.location.href = '/login?reason=expired'; } return Promise.reject(transformError(error)); }); // Error transformer const transformError = (error: AxiosError): ApiError => { if (error.response?.data) { const data = error.response.data as any; return { message: data.message || 'Error en la solicitud', status: error.response.status, errors: data.errors, }; } if (error.request) { return { message: 'Error de red. Verifica tu conexión.', status: 0, }; } return { message: error.message || 'Error desconocido', }; }; // Generic API call helper interface ApiCallConfig { method: 'GET' | 'POST' | 'PUT' | 'DELETE' | 'PATCH'; url: string; data?: any; params?: any; } export const apiCall = async <T>(config: ApiCallConfig): Promise<T> => { const response = await apiClient.request<T>({ method: config.method, url: config.url, data: config.data, params: config.params, }); return response.data; };
```

## Paso 4: Servicios (api/services/auth.service.ts)

typescript

```
import { apiCall } from '../client'; import { setAuthToken } from '../client'; import { LoginRequest, RegisterRequest, AuthResponse, LogoutResponse, User, } from '../types/auth.types'; import { AUTH_CONFIG } from '@config/auth.config'; export const authService = { login: async (credentials: LoginRequest): Promise<AuthResponse> => { const response = await apiCall<AuthResponse>({ method: 'POST', url: AUTH_CONFIG.ENDPOINTS.login, data: credentials, }); setAuthToken(response.access_token); return response; }, register: async (userData: RegisterRequest): Promise<AuthResponse> => { const response = await apiCall<AuthResponse>({ method: 'POST', url: AUTH_CONFIG.ENDPOINTS.register, data: userData, }); setAuthToken(response.access_token); return response; }, logout: async (): Promise<LogoutResponse> => { try { const response = await apiCall<LogoutResponse>({ method: 'POST', url: AUTH_CONFIG.ENDPOINTS.logout, }); return response; } finally { removeAuthToken(); } }, me: async (): Promise<User> => { return apiCall<User>({ method: 'GET', url: AUTH_CONFIG.ENDPOINTS.me, }); }, };
```



## Paso 5: Contexto (contexts/AuthContext.tsx)

typescript

```
import React, { createContext, useContext, useState, useEffect, ReactNode } from 'react'; import { authService } from '@api/services/auth.service'; import { getAuthToken, removeAuthToken } from '@api/client'; import { User, LoginRequest, RegisterRequest } from '@api/types/auth.types'; interface AuthContextValue { user: User | null; isAuthenticated: boolean; isLoading: boolean; login: (credentials: LoginRequest) => Promise<void>; register: (userData: RegisterRequest) => Promise<void>; logout: () => Promise<void>; updateUser: (user: User) => void; } const AuthContext = createContext<AuthContextValue | undefined>(undefined); interface AuthProviderProps { children: ReactNode; } export const AuthProvider: React.FC<AuthProviderProps> = ({ children }) => { const [user, setUser] = useState<User | null>(null); const [isLoading, setIsLoading] = useState(true); useEffect(() => { const initializeAuth = async () => { const token = getAuthToken(); if (token) { try { const userData = await authService.me(); setUser(userData); } catch (error) { console.error('Token validation failed:', error); removeAuthToken(); } } setIsLoading(false); }, [initializeAuth]; }, []); const login = async (credentials: LoginRequest): Promise<void> => { setIsLoading(true); try { const response = await authService.login(credentials); setUser(response.user); const redirectPath = localStorage.getItem('redirect_after_login'); if (redirectPath) { localStorage.removeItem('redirect_after_login'); window.location.href = redirectPath; } } finally { setIsLoading(false); } } const register = async (userData: RegisterRequest): Promise<void> => { setIsLoading(true); try { const response = await authService.register(userData); setUser(response.user); } finally { setIsLoading(false); } } const logout = async (): Promise<void> => { setIsLoading(true); try { await authService.logout(); } finally { setUser(null); setIsLoading(false); } } const updateUser = (updatedUser: User) => { setUser(updatedUser); } return (<AuthContext.Provider value={{ user, isAuthenticated: !!user, isLoading, login, register, logout, updateUser, }} > {children} </AuthContext.Provider>); } export const useAuth = (): AuthContextValue => { const context = useContext(AuthContext); if (!context) { throw new Error('useAuth must be used within AuthProvider'); } return context; }
```

## Paso 6: Sistema de Roles (hooks/useRole.ts)

typescript

```
import { useAuth } from '@contexts/AuthContext'; import { UserRole } from '@api/types/auth.types'; const roleHierarchy: Record<UserRole, number> = { 'User': 1, 'Moderator': 2, 'Admin': 3, }; interface RolePermissions { canCreateDisciplines: boolean; canEditOwnDisciplines: boolean; canEditAnyDiscipline: boolean; canViewUserStats: boolean; canManageUsers: boolean; canDeleteUsers: boolean; canViewAdminPanel: boolean; } export const useRole = () => { const { user, isAuthenticated } = useAuth(); const hasMinimumRole = (requiredRole: UserRole): boolean => { if (!isAuthenticated || !user?.role) return false; return roleHierarchy[user.role] >= roleHierarchy[requiredRole]; } const hasExactRole = (role: UserRole): boolean => { if (!isAuthenticated || !user?.role) return false; return user.role === role; } const permissions: RolePermissions = { canCreateDisciplines: hasMinimumRole('User'), canEditOwnDisciplines: hasExactRole('User'), canEditAnyDiscipline: hasMinimumRole('Moderator'), canViewUserStats: hasMinimumRole('Moderator'), canManageUsers: hasMinimumRole('Moderator'), canDeleteUsers: hasMinimumRole('Admin'), canViewAdminPanel: hasMinimumRole('Admin'), }; return { user, userRole: user?.role || null, permissions, hasMinimumRole, hasExactRole, }; }
```

## Paso 7: Timeout de Inactividad (hooks/useInactivityLogout.ts)

typescript

```
import { useEffect, useCallback, useRef } from 'react'; import { useAuth } from '@contexts/AuthContext'; import { AUTH_CONFIG } from '@config/auth.config'; export const useInactivityLogout = (timeoutMs: number = AUTH_CONFIG.SECURITY.INACTIVITY_TIMEOUT) => { const { isAuthenticated, logout } = useAuth(); const timeoutRef = useRef<NodeJS.Timeout | null>(null); const resetTimer = useCallback(() => { if (timeoutRef.current) { clearTimeout(timeoutRef.current); } if (isAuthenticated) { timeoutRef.current = setTimeout(() => { console.warn('Session expired due to inactivity'); logout(); window.location.href = '/login?reason=inactivity'; }, timeoutMs); }, [isAuthenticated, logout, timeoutMs]); useEffect(() => { if (!isAuthenticated) return; const events = ['mousedown', 'keydown', 'scroll', 'touchstart', 'click']; events.forEach(event => { document.addEventListener(event, resetTimer, true); }); resetTimer(); return () => { if (timeoutRef.current) { clearTimeout(timeoutRef.current); } events.forEach(event => { document.removeEventListener(event, resetTimer, true); }); }, [isAuthenticated, resetTimer]; }
```



## Componentes UI

### AuthForm (components/Auth/AuthForm.tsx)

typescript

```
import React, { useState } from 'react';import { useAuth } from '@/contexts/AuthContext';import { ApiError } from '@/api/types/auth.types';interface AuthFormProps { mode?: 'login' | 'register'; onSuccess?: () => void; }export const AuthForm: React.FC<AuthFormProps> = ({ mode = 'login', onSuccess }) => { const { login, register, isLoading } = useAuth(); const [isLoginMode, setIsLoginMode] = useState(mode === 'login'); const [error, setError] = useState<string | null>(null); const [formData, setFormData] = useState({ name: '', email: '', password: '', password_confirmation: '' }); const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => { setFormData(prev => ({ ...prev, [e.target.name]: e.target.value, })); setError(null); const handleSubmit = async (e: React.FormEvent) => { e.preventDefault(); setError(null); try { if (isLoginMode) { await login({ email: formData.email, password: formData.password, }); } else { if (formData.password !== formData.password_confirmation) { setError('Las contraseñas no coinciden'); return; } await register(formData); } onSuccess?.(); } catch (err: any) { const apiError = err as ApiError; if (apiError.errors) { const firstError = Object.values(apiError.errors)[0]; setError(firstError[0]); } else { setError(apiError.message || 'Error de autenticación'); } } } return (<form onSubmit={handleSubmit} className="auth-form"> {error && (<div className="error-message" role="alert"> {error} </div>)} {!isLoginMode && (<div className="form-group"> <label htmlFor="name">Nombre</label> <input type="text" id="name" name="name" value={formData.name} onChange={handleChange} required disabled={isLoading} /> </div>)} <div className="form-group"> <label htmlFor="email">Email</label> <input type="email" id="email" name="email" value={formData.email} onChange={handleChange} required disabled={isLoading} /> </div> <div className="form-group"> <label htmlFor="password">Contraseña</label> <input type="password" id="password" name="password" value={formData.password} onChange={handleChange} required minLength={8} disabled={isLoading} /> </div> {!isLoginMode && (<div className="form-group"> <label htmlFor="password_confirmation">Confirmar Contraseña</label> <input type="password" id="password_confirmation" name="password_confirmation" value={formData.password_confirmation} onChange={handleChange} required minLength={8} disabled={isLoading} /> </div>)} <button type="submit" disabled={isLoading} className="submit-button"> {isLoading ? 'Procesando...' : (isLoginMode ? 'Iniciar Sesión' : 'Registrarse')} </button> <button type="button" onClick={() => { setIsLoginMode(!isLoginMode); setError(null); }} className="toggle-mode-button" disabled={isLoading} > {isLoginMode ? '¿No tienes cuenta? Regístrate' : '¿Ya tienes cuenta? Inicia sesión'} </button> </form>);};
```

### RoleGuard (components/RoleGuard/RoleGuard.tsx)

typescript

```
import React from 'react';import { useAuth } from '@/contexts/AuthContext';import { useRole } from '@/hooks/useRole';import { UserRole } from '@/api/types/auth.types';interface RoleGuardProps { children: React.ReactNode; allowedRoles?: UserRole[]; minimumRole?: UserRole; requireAuth?: boolean; fallback?: React.ReactNode; }export const RoleGuard: React.FC<RoleGuardProps> = ({ children, allowedRoles, minimumRole, requireAuth = true, fallback = null, }) => { const { isAuthenticated } = useAuth(); const { user, hasMinimumRole } = useRole(); if (requireAuth && !isAuthenticated) { return <>{fallback}</> } if (allowedRoles && user?.role && !allowedRoles.includes(user.role)) { return <>{fallback}</> } if (minimumRole && !hasMinimumRole(minimumRole)) { return <>{fallback}</> } return <>{children}</>; } // Componentes especializadosexport const AdminOnly: React.FC<{ children: React.ReactNode; fallback?: React.ReactNode }> = ({ children, fallback = null }) => (<RoleGuard allowedRoles={['Admin']} fallback={fallback}> {children} </RoleGuard>);export const ModeratorPlus: React.FC<{ children: React.ReactNode; fallback?: React.ReactNode }> = ({ children, fallback = null }) => (<RoleGuard minimumRole="Moderator" fallback={fallback}> {children} </RoleGuard>);export const AuthenticatedOnly: React.FC<{ children: React.ReactNode; fallback?: React.ReactNode }> = ({ children, fallback = null }) => (<RoleGuard requireAuth={true} fallback={fallback}> {children} </RoleGuard>);
```

## Integración en App.tsx

typescript

```
import React, { useEffect } from 'react';import { QueryClient, QueryClientProvider } from '@tanstack/react-query';import { AuthProvider } from './contexts/AuthContext';import { useAuth } from './contexts/AuthContext';import { useInactivityLogout } from './hooks/useInactivityLogout';import { validateAuthConfig } from './config/auth.config';import { AuthForm } from
```

```

'./components/Auth/AuthForm';import Dashboard from './pages/Dashboard';const queryClient = new
QueryClient({ defaultOptions: { queries: { staleTime: 60 * 1000, retry: 2, refetchOnWindowFocus:
false, }, }, });function App() { useEffect(() => { validateAuthConfig(); }, []); return (<QueryClientProvider
client={queryClient}> <AuthProvider> <AppContent /> </AuthProvider> </QueryClientProvider>);}function
AppContent() { const { isAuthenticated, isLoading } = useAuth(); // Activar timeout de inactividad
useInactivityLogout(); if (isLoading) { return <div className="loading">Cargando...</div>; } return (<div
className="app"> {isAuthenticated ? <Dashboard /> : <AuthForm />} </div>);}export default App;

```

## Uso en Componentes

### Ejemplo: Dashboard con Guards

typescript

```

import React from 'react';import { useAuth } from '@contexts/AuthContext';import { useRole } from
'@hooks/useRole';import { AdminOnly, ModeratorPlus, AuthenticatedOnly } from
'@components/RoleGuard/RoleGuard';export const Dashboard: React.FC = () => { const { user, logout } =
useAuth(); const { permissions } = useRole(); const handleLogout = async () => { try { await logout();
window.location.href = '/login'; } catch (error) { console.error('Logout failed:', error); } }; return (<div
className="dashboard"> <header> <h1>Bienvenido, {user?.name}</h1> <button
onClick={handleLogout}>Cerrar Sesión</button> </header> <nav> <AuthenticatedOnly> Disciplinas Comunidades </AuthenticatedOnly>
<ModeratorPlus> Moderación Usuarios </ModeratorPlus>
<AdminOnly> Administración </AdminOnly> </nav>
<main>{permissions.canCreateDisciplines && (<button>Crear Disciplina</button>)} <ModeratorPlus>
<section> <h2>Estadísticas</h2> { /* Contenido solo para Moderator+ */ } </section> </ModeratorPlus>
</main> </div>);};

```

## Testing

### Setup de Tests

typescript

```

// setupTests.tsimport '@testing-library/jest-dom';import { QueryClient, QueryClientProvider } from
'@tanstack/react-query';export const createTestQueryClient = () => new QueryClient({ defaultOptions: { queries:
{ retry: false, }, mutations: { retry: false, }, }, });export const wrapper = ({ children }: { children:
React.ReactNode }) => (<QueryClientProvider client={createTestQueryClient()}> {children}
</QueryClientProvider>);

```

### Test de Login

typescript

```

import { renderHook, waitFor } from '@testing-library/react';import { useAuth } from
'@contexts/AuthContext';import { AuthProvider } from '@contexts/AuthContext';describe('useAuth', () => {
it('should login successfully', async () => { const wrapper = ({ children }: any) => (
<AuthProvider>{children}</AuthProvider>); const { result } = renderHook(() => useAuth(), { wrapper }); await
act(async () => { await result.current.login({ email: 'test@example.com', password: 'password123', }); });
expect(result.current.isAuthenticated).toBe(true); expect(result.current.user).toBeDefined(); });});

```

## Activación del Sistema XP (Futuro)

### Paso 1: Verificar Endpoints Backend

bash

```

Verificar que estos endpoints existan:curl http://localhost:8000/api/v1/users/1/levelcurl
http://localhost:8000/api/v1/leaderboard

```

### Paso 2: Descomentar Código

typescript

```
// components/Layout/Header.tsx// ✗ ELIMINAR mock:// const userLevel = 12;// const userXp = 2450;//
DESCOMENTAR:import { UserLevelDisplay } from '@components/UserLevel/UserLevelDisplay';// En
JSX:<UserLevelDisplay size="small" />
```

### Paso 3: Agregar Ruta

typescript

```
// App.tsx o Router.tsximport { LeaderboardPage } from '@pages/LeaderboardPage';<Route path="/leaderboard"
element={ <AuthenticatedOnly> <LeaderboardPage /> </AuthenticatedOnly> } />
```

### Troubleshooting

#### Error: "CORS policy"

**Solución:** Configurar CORS en backend Laravel:

php

```
// config/cors.php'paths' => ['api/*'],'allowed_origins' => ['http://localhost:3000'],'allowed_methods' =>
['*'],'allowed_headers' => ['*'],
```

#### Error: "401 Unauthorized"

##### Diagnóstico:

Verificar token en localStorage

1. 2.

Verificar header Authorization en network tab

Verificar que el token no haya expirado

3.

**Solución:**

typescript

```
// Limpiar y relógearlocalStorage.clear();window.location.href = '/login';
```

#### Error: "Network Error"

##### Diagnóstico:

Verificar que backend esté corriendo

Verificar URL en .env.local

Verificar firewall/antivirus

2. 3.

### Checklist de Producción

Variables de entorno configuradas

HTTPS habilitado

Timeout de inactividad probado

Errores manejados correctamente

Tests pasando al 100%

CORS configurado

Rate limiting verificado

Logging configurado

Performance optimizada

Documentación actualizada

### Referencias

**Código completo:** Artefactos generados anteriormente

**React Query Docs:** <https://tanstack.com/query/latest>

**Axios Docs:** <https://axios-http.com/>

**TypeScript Handbook:** <https://www.typescriptlang.org/docs/>

1.

**Documento generado:** 15 de octubre de 2025 **Versión:** 1.0 **Estado:** ☒ Listo para implementación **Soporte:** Ver código fuente en artefactos

El primer conflicto viene de ahí. La API backend sólo se soporta con un token. Un token "Bearer" que se entrega al loggearse, y se destruye o se da fecha de expiración, al hacer logout. Por lo tanto, no sigue el protocolo OAuth2, el endpoint logout es custom y no atañe a la funcionalidad integrada de Passport. Por lo que, estoy redactando .mds de cambios realizados, diseño y toda la información frontend, proporcionada Con un React Assistant GPT de OpenAI y Laravel GPT para intercambio de documentación entre ambas partes. El intercambio simula una reunión entre agentes front y back. En cuanto a diseño, se encarga Claude Sonnet.

(Vease pdf numero 6 – Integration Report Laravel Api)

Una vez que se entiendan y se aclaren, frontend realizará los cambios oportunos, mientras se acaba de crear todas las vistas del proyecto. Para ello he seguido usando metodología ASPECCT.

## GUIA / RESUMEN DE PROMPTS CON MIS AGENTES PARA COMPATIBILIDAD E INTEGRACIÓN FRONTEND/BACKEND

Paso 1 - Sonnet - Crear documentación con las incongruencias a resolver con el backend. (Ya proporcionado)

Paso 2 - Copilot - Añadir documentacion y consultar modo Ask que contraste la información y que verifique los protocolos usados dada la documentación ya existente.

Importante: Backend no tiene que realizar cambios, frontend tiene que adaptarse a lo que hay, y informar qué cambios tiene que realizar, generando un informe de guía para el frontend con todos los puntos comentados de la autenticación se refiere.

Paso 3 - Laravel GPT - Documentar los 2 pasos anteriores y que confirme lo hablado, reforzando los pasos que front ha de asumir. Redactar la confirmación

Paso 4 - React Assistant GPT - Con todos los informes previos de backend, pedir que realice los cambios de implementación en TypeScript para el Auth y que genere el código

de toda la implementación y documentación.

Paso 5 - Copilot - Adaptación del frontend al backend.

Otra opción es que, teniendo la confirmación del mentor acerca del token, puedo especificar.

---UPDATE---

Mejoras para la interacción CRUD que el boton fab "+" sirva para que abra un modal sobre qué quiere crear:

Un usuario, una disciplina, o una comunidad.

Actualizar el listado de comunidades igual que el de disciplinas

Listado de usuarios en el aside lateral de usuarios.

Que al clicar a un usuario del listado de usuarios aparezca su info personal

NO TIENE CONFIG DE MOMENTO

NO HAY REPORTES DE MOMENTO

EN MODERACION MUESTRA Y DEBE MOSTRAR LA POSIBILIDAD DE ASIGNAR UN ROL MODERADOR A UN

USUARIO QUE PERTENEZCA A UNA COMUNIDAD

EN ANALYTICS, QUE MUESTRE LAS STATS DE TODO EL CONTENIDO.

APLICAR GLAFICAS, TABLES DE QUESO, BLABLABLA

valorar un div tanstack query integrada en el aside.

También, en la parte de gestionar disciplinas, al seleccionar una disciplina, deberías ver las estadísticas de esa disciplina, actualizar disciplina (name/description) y borrar,

dando lugar al mismo modal de confirmacion de borrado.

En comunidades activas, deberia ver las stats completas de la comunidad al clicar

El span del batch de cada comunidad activa corresponde a los usuarios activos.

La cantidad de miembros son los que contiene esa comunidad.

Al clicar en el texto X miembros, redirigir al listado de usuarios